Department of Computer Science

Software Engineering

Final Project (30%)

Phase 2: Software Design Specification Document (10%)

Rihla

Desert and City Experience Booking Platform for Dubai

Software Design Specification (SDS)

Team members and parts

Salem Al Tamimi: Sections 1 and 2

Hamzah and Hamad: Section 3

Mir Umer: Section 4

Omar Arif: Section 5

Oumar Ibrahim: Section 6

Implementation: Python 3.12, Tkinter client, Flask REST server, SQLite database

All facts in this document were checked against the project source code (repository rihla_manual, branch main).

Table of Contents

1.0 Introduction 4
1.1 Goals and objectives 4
1.2 Statement of scope 4
1.3 Software context 5
1.4 Major constraints 6
2.0 Data design 7
2.1 Internal software data structure 7
2.2 Global data structure 8
2.3 Temporary data structure 8
2.4 Database description 8
3.0 Architectural and component-level design 13
3.1 System Structure 13
3.2 Description of components (overview) 17
3.2 Description for Component 1: Authentication and Accounts 18
3.2 Description for Component 2: Event and Venue Management 20
3.2 Description for Component 3: Booking, Pricing and Payment 21
3.2 Description for Component 4: Cancellation and Refund 24
3.2 Description for Component 5: Venue Operations (Staff) 25
3.2 Description for Component 6: Fleet and Dispatch 27
3.2 Description for Component 7: Administration and Reports 28
3.2 Description for Component 8: Reviews and Support Tickets 30
3.3 Dynamic behavior of the components 31
4.0 User interface design 39
4.1 Description of the user interface 39
4.2 Interface design rules 48
4.3 Components available 49
4.4 UIDS description 49
5.0 Restrictions, limitations, and constraints 50
6.0 Testing Issues 51
6.1 Classes of tests 51

6.2 Expected software response 52
6.3 Performance bounds 52
6.4 Identification of critical components 52

1.0 Introduction

This Software Design Specification (SDS) describes how Rihla is built. Rihla is a desktop application for booking desert safaris, dinners, yacht trips and other experiences in Dubai. In Phase 1 we wrote the requirements. In this phase we explain the data design, the architecture, the components, the user interface and the tests of the software that we really implemented.

Our Phase 1 plan was to build Rihla as a custom module on top of Odoo with PostgreSQL. During the semester we changed this plan. The final product is a Python program with two parts: a Tkinter desktop client and a small Flask web server that owns one SQLite database. We made this change because the whole team knows Python well, Tkinter ships with Python, and a small server we write ourselves is easier to test and to explain than a large framework. This document describes the final design, not the Phase 1 plan.

1.1 Goals and objectives

The main goal of Rihla is to connect tourists with local experience providers and to give every person in the process a clear screen for their own job. A tourist should find an experience, see how many seats are left, pay, and get a ticket code in a few steps. The provider, the venue staff, the transport team, travel agencies and the platform administrator should each work with the same live data, so seats are never sold twice.

Our objectives are:

- **Safe seat booking:** check capacity on the server inside one database transaction, so two people cannot book the last seats at the same time.
- **Clear prices:** add the mandatory 5% UAE VAT to every booking, keep all money in AED, and show prices in AED, USD, EUR or GBP for the user.
- **Digital tickets:** give every booking a unique 16-digit ticket code that venue staff can check in only once, and a booking receipt the user can save as a PDF.
- **Fair cancellations:** apply one written refund policy for everybody and record who cancelled, when, why and how much was refunded.
- **Role-based work:** give each of the six roles (Customer, Vendor, Staff, Partner, Dispatcher, Admin) its own home screen and its own server rules.
- **Trust and safety:** require a 16-digit SSN for every non-customer account, let users report problems, and let the admin ban accounts.

1.2 Statement of scope

Rihla covers the full life of a booking: an event is created by a vendor, transport is assigned by a dispatcher, a customer or partner books and pays, staff check the ticket in at the venue, and the customer can review the experience afterwards. Cancellations, refunds, support questions, reports and bans are part of the scope.

Major inputs are account details, event details (title, category, price in AED, venue, date or weekly days, time, capacity and a JPEG picture), booking requests (event, date, guests), payment details, ticket codes, vehicle data, support messages and reports. Major outputs are event lists with live capacity, booking confirmations with ticket codes, VAT breakdowns, PDF receipts, check-in results, daily venue schedules, refund amounts and admin tables.

The table below ranks the main functions from the developer's point of view. Essential functions are finished and tested. Desirable functions were planned and are partly done. Future functions are not in the current code.

Function	Priority	Status in the code
Sign up, log in, password reset, six roles	Essential	Done. Passwords use salted PBKDF2.
16-digit SSN for non-customer accounts	Essential	Done. Only a hash and the last 4 digits are stored.
Event, venue and category management (vendor)	Essential	Done, with pictures and weekly recurrence.
Explore, search and filter tours	Essential	Done (category, maximum price, text search).
Capacity check per date	Essential	Done. Vehicle seats or the event capacity.
Booking with 5% VAT and currency display	Essential	Done. Totals are stored in AED with VAT.
Demo payment (Visa, Mastercard, bank transfer)	Essential	Done. Checked against fixed demo records.
Booking Details window and PDF receipt	Essential	Done. PDF written without extra libraries.
Ticket check-in by venue staff	Essential	Done. Scoped to the staff member's venue.
Cancellation and refund policy	Essential	Done. 100%, 50% or refused, with an audit trail.
Partner shopping list and one-payment checkout	Essential	Done. All items in one transaction.
Vehicle registration and event dispatch	Essential	Done. Assigned seats become the capacity.
Admin: accounts, bans, reports, all bookings	Essential	Done.
Reviews and average ratings	Desirable	Done (0 to 5 in steps of 0.5).
Support tickets answered by venue staff	Desirable	Done, scoped to the venue.
Google Maps link for venues	Desirable	Done (only Google Maps https links).
Real email and SMS verification	Desirable	Not done. The code is always 000000.
Real payment gateway and real refunds	Future	Not done. Payments and refunds are demo records.
QR codes, offline driver app, live vehicle map	Future	Not done.
Arabic and other languages	Future	Not done. The interface is English only.

Table 1: Ranking of the major functions

1.3 Software context

Tourism is one of the biggest industries in Dubai. Many small operators still take bookings by phone and chat, and they lose track of seats when bookings come from walk-in tourists, hotels and travel agencies at the same time. Rihla puts all these channels on one system. Customers book for themselves. Partners (travel agencies) book groups for their clients and can book more than 10 guests at once. Vendors see every booking for their events. Dispatchers decide how many seats an event has by assigning vehicles. Staff at the venue see the schedule of the day and admit guests.

Technically, Rihla is a client and server system. The client (folder src/) is a Tkinter desktop program that the user starts with start_client.py. It never opens the database. Every read and write goes through api_client.py, which sends HTTP requests with JSON to the Flask server (folder server/). The server listens on 127.0.0.1 port 5000 and is the only program that opens server/users.db. This keeps all business rules (capacity, VAT, refunds, bans, venue scope) in one place. Figure 1 shows this structure.

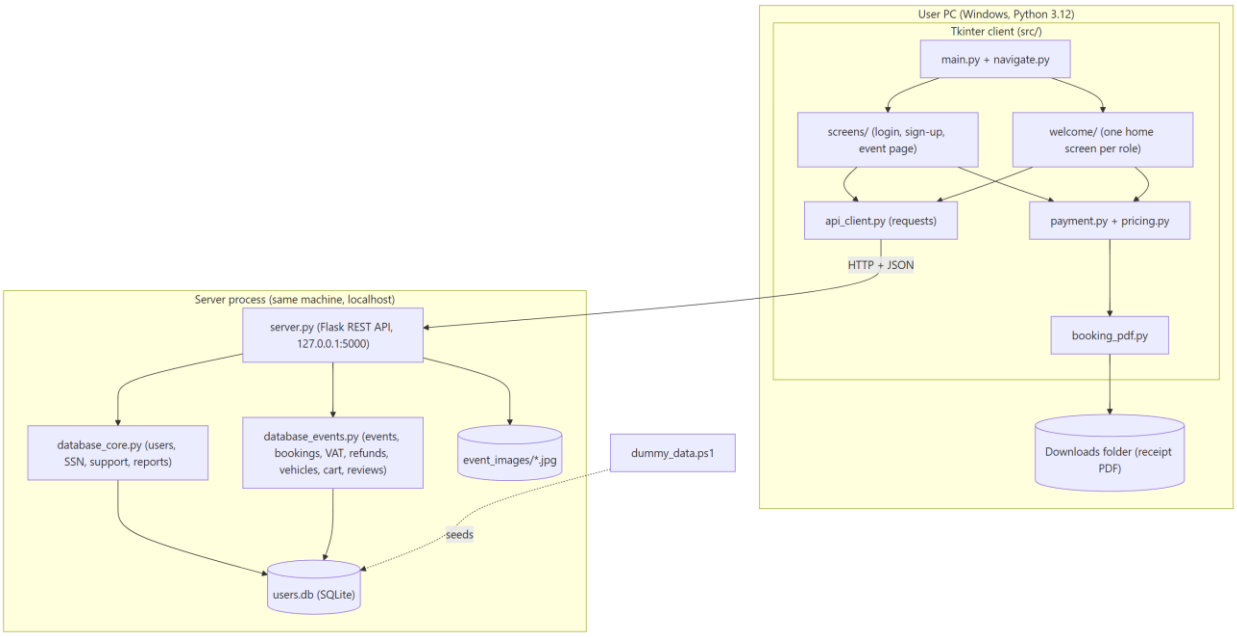


Figure 1: Component and deployment view of Rihla

1.4 Major constraints

- **Time and team size:** the project had to be finished in one semester by a student team, so we chose tools we already knew (Python, Tkinter, Flask, SQLite) and avoided large frameworks.
- **UAE tax rules:** VAT is 5% and must be shown as a separate line. Money is kept in AED. Other currencies are for display only.
- **Platform:** the client targets Windows 10 or 11 with Python 3.12. Some details are Windows specific, for example finding the Downloads folder through the Windows registry.
- **No real payments:** we may not process real cards in a course project, so payment is a demo that compares the form with fixed demo details.
- **Privacy:** an SSN is sensitive data, so the server stores only a salted hash and the last 4 digits.
- **Demo readiness:** the project must be easy to show, so the server creates six demo accounts at start and dummy_data.ps1 fills the database with realistic data relative to today.

2.0 Data design

This section describes the data that moves inside Rihla. It covers the data passed between components, the data shared by large parts of the program, temporary files, and the database.

2.1 Internal software data structure

Components exchange plain Python dictionaries and lists. Between the client and the server the same dictionaries travel as JSON. Every server answer has a success flag, and a failed answer also has a message that the client shows to the user without changes. The most important structures are:

Structure	Main fields	Produced by and used by
Event	id, title, category, price, desc, vendor_email, venue_id, venue_name, maps_url, event_datetime, is_recurring, recur_day, recur_time, capacity	database_events.get_all_events to the explore, event and vendor screens
Capacity info	total, source ('vehicles' or 'event'), vehicles, booked, remaining	_capacity_info to the event page and the CapacityBar widget
Booking	id, event_id, event_title, venue_name, booking_date, event_time, guests, total_price, ticket_code, status, can_cancel, refund_if_cancelled, cancelled_by, cancel_reason, refund_percent, refund_amount	_booking_dict to My Bookings, vendor, partner and admin tables
Booking result	success, message, booking (with base_price and vat)	create_booking to payment.show_booking_details_window
Payment	txn_id, method, currency	payment modal to the Booking Details window and PDF
Receipt lines	list of (text, font size, bold)	payment._receipt_lines to the window and booking_pdf
Refund	percent, base, vat, total	_cancel to the cancel dialogs
Cart item	id, event_id, booking_date, guests, client_name, title, unit_price, total_price	get_cart_items to the partner shopping list
Vehicle assignment	vehicle_id, auto_reassign_recurring	dispatch view to dispatch_vehicle_combination
User row (admin)	email, role, created_at, banned, banned_at, banned_by, venue_name, manager_email	get_all_users to the admin User Accounts table

Table 2: Data structures passed between components

Booking statuses are computed when the booking is read. The database keeps status 'Confirmed' or 'Cancelled' in bookings and 'Valid', 'Used' or 'Cancelled' in tickets. The function _booking_dict joins both and shows 'Cancelled', 'Checked in', 'Expired' (date passed) or 'Confirmed'.

2.2 Global data structure

The main global data store is the SQLite database server/users.db. Every component reads and writes it only through the server functions in database_core.py and database_events.py. Next to it, the server keeps event pictures as files in server/event_images/<event id>.jpg.

Some constants are shared by large parts of the program:

- **config.py** (one copy in src/ and one in server/): the six roles, the demo accounts, SIGNUP_ROLES (all roles except Admin) and MIN_PASSWORD_LENGTH = 8.
- **pricing.py** (client): VAT_RATE = 0.05, the display rates for 1 AED (USD 0.2723, EUR 0.2510, GBP 0.2150) and the money formatting functions.
- **database_events.py** (server): VAT_RATE = 0.05, FULL_REFUND_HOURS = 48, HALF_REFUND_HOURS = 24, MAX_CANCEL_REASON = 300.
- **database_core.py** (server): DEFAULT_EVENT_CAPACITY = 20, MAX_EVENT_CAPACITY = 5000, MAX_GUESTS_PER_BOOKING = 10, MAX_OPEN_SUPPORT_TICKETS = 3, SSN_LENGTH = 16.
- **theme.py** (client): shared colours and the Segoe UI font, so every screen looks the same.

On the client, each home screen object keeps the signed-in email and role in memory while the window is open. There is no other global session state.

2.3 Temporary data structure

- **Partner cart:** the partner_cart table holds events a partner wants to book. Rows are deleted after a successful checkout, or when the partner removes them.
- **Image cache:** the client keeps downloaded event pictures in memory (event_images.py) and forgets one when the vendor changes it.
- **Uploaded picture:** when a vendor picks a picture, the client shrinks it to at most 1400 pixels, encodes it as base64 JPEG and sends it in the JSON body. The server checks it (JPEG, at most 4 MB) and writes it as a file.
- **Receipt PDF:** booking_pdf.py writes Rihla_Booking_<transaction id>.pdf to the user's Downloads folder. It never overwrites an older file. It adds (1), (2) and so on to the name.
- **Test databases:** the unit tests point DB_PATH to a temporary file, so server/users.db is never touched by tests.
- **Demo seed table:** dummy_data.ps1 records every row it adds in a demo_seed table, so the next run (or -Clear) can remove exactly those rows.

2.4 Database description

Rihla uses one SQLite database. The function init_db in database_core.py creates all tables with CREATE TABLE IF NOT EXISTS, adds missing columns to older databases with ALTER TABLE, and inserts default data (vehicle types, two venues, categories, two demo events, seven vehicles, the demo ticket and the six demo accounts). The reviews table is created the first time it is needed. Figure 2 shows the entities and their

relationships. Many links use an email address (for example customer_email or vendor_email) instead of a foreign key, because the users table has no integer id.

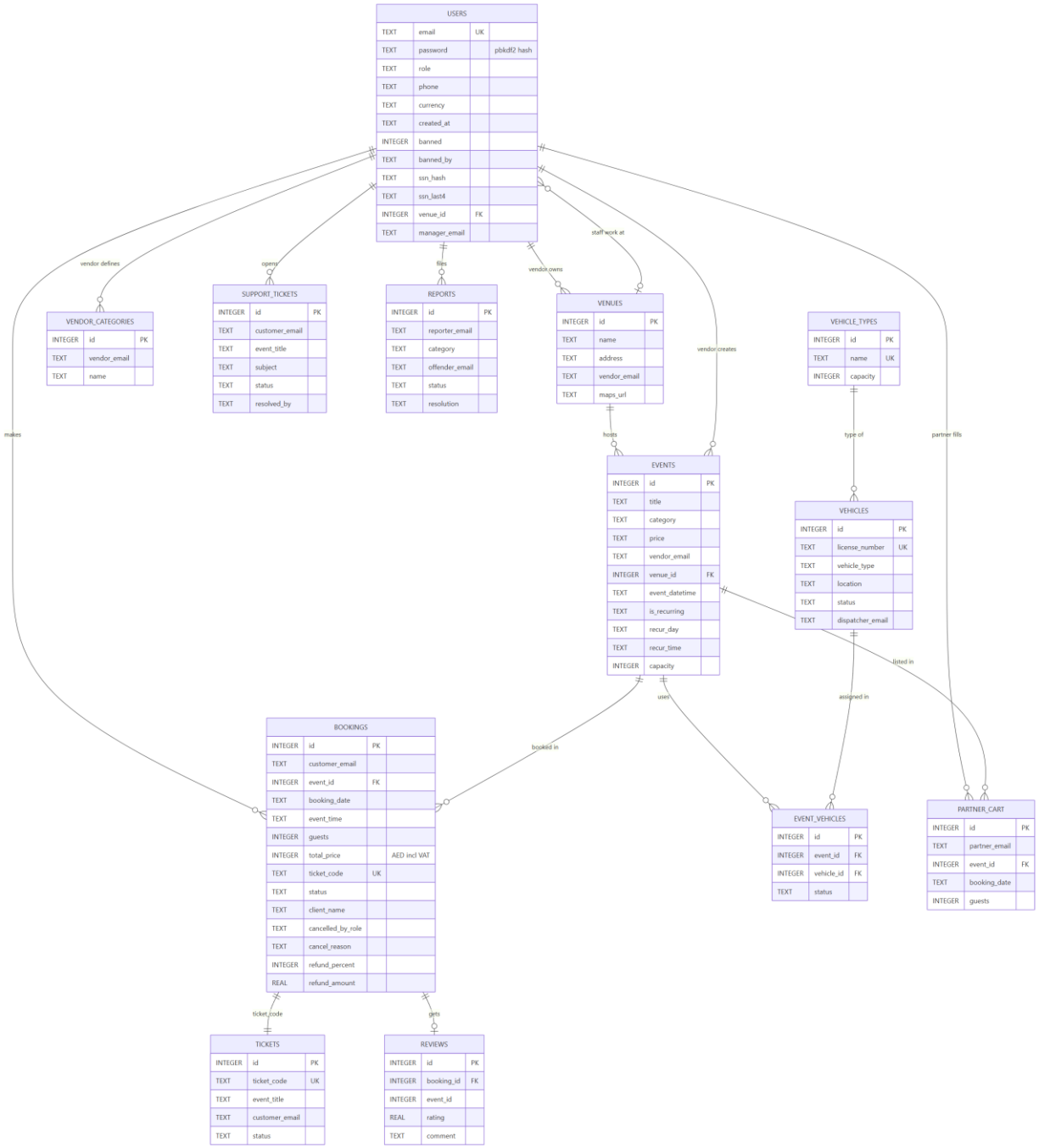


Figure 2: Entity relationship diagram of server/users.db

Table users. One row per account.

Column	Type	Constraint or meaning
email	TEXT	UNIQUE. Compared without letter case.
password	TEXT	pbkdf2$200000$salt$hash (salted PBKDF2-SHA256)

Column	Type	Constraint or meaning
role	TEXT	NOT NULL, default 'Customer'
phone	TEXT	UAE mobile number
currency	TEXT	AED, USD, EUR or GBP, default AED
created_at	TEXT	sign-up time (server local time)
banned, banned_at, banned_by	INTEGER, TEXT, TEXT	banned is 0 or 1, default 0
ssn_hash, ssn_last4	TEXT, TEXT	only for non-customer roles
venue_id	INTEGER	REFERENCES venues(id), staff only
manager_email	TEXT	staff only, stored text, no account

Table 3: Table users

Table venues. Places where events happen.

Column	Type	Constraint or meaning
id	INTEGER	PRIMARY KEY AUTOINCREMENT
name, address	TEXT	name NOT NULL
vendor_email	TEXT	owner vendor
maps_url	TEXT	NOT NULL default '', Google Maps https link

Table 4: Table venues

Table events. Experiences that can be booked.

Column	Type	Constraint or meaning
id	INTEGER	PRIMARY KEY AUTOINCREMENT
title	TEXT	NOT NULL
category, price, description	TEXT	price text such as '240 AED'
vendor_email	TEXT	owner vendor
venue_id	INTEGER	FOREIGN KEY venues(id)
event_datetime	TEXT	'YYYY-MM-DD HH:MM' for one-time events
is_recurring, recur_day, recur_time	TEXT	'Recurring' or 'One-Time', weekdays such as 'Tuesday, Thursday'
capacity	INTEGER	NOT NULL default 20, 1 to 5000

Table 5: Table events

Table bookings. One row per paid booking.

Column	Type	Constraint or meaning
id	INTEGER	PRIMARY KEY AUTOINCREMENT
customer_email	TEXT	NOT NULL, the customer or partner who booked
event_id	INTEGER	FOREIGN KEY events(id)
event_title, venue_name	TEXT	copies kept for history
booking_date, event_time	TEXT	booking_date NOT NULL
guests	INTEGER	NOT NULL default 1
total_price	INTEGER	NOT NULL, AED including VAT
ticket_code	TEXT	UNIQUE NOT NULL, 16 digits
status	TEXT	'Confirmed' or 'Cancelled'
client_name	TEXT	partner's client group
created_at	TEXT	default CURRENT_TIMESTAMP

Column	Type	Constraint or meaning
cancelled_at, cancelled_by, cancelled_by_role, cancel_reason	TEXT	cancellation audit trail
refund_percent, refund_amount	INTEGER, REAL	100 or 50, amount in AED

Table 6: Table bookings

Table tickets. Entry tickets checked by staff.

Column	Type	Constraint or meaning
id	INTEGER	PRIMARY KEY AUTOINCREMENT
ticket_code	TEXT	UNIQUE
event_title, customer_email	TEXT	
status	TEXT	'Valid', 'Used' or 'Cancelled'

Table 7: Table tickets

Table reviews. Ratings of past experiences.

Column	Type	Constraint or meaning
id	INTEGER	PRIMARY KEY AUTOINCREMENT
booking_id	INTEGER	NOT NULL UNIQUE, FOREIGN KEY bookings(id)
event_id, customer_email	INTEGER, TEXT	index on event_id
rating	REAL	0 to 5 in steps of 0.5
comment	TEXT	at most 500 characters
created_at, updated_at	TEXT	default CURRENT_TIMESTAMP

Table 8: Table reviews

Table support_tickets. Questions from customers to venue staff.

Column	Type	Constraint or meaning
id	INTEGER	PRIMARY KEY AUTOINCREMENT
customer_email, subject, message	TEXT	NOT NULL
event_title	TEXT	the event the question is about
status	TEXT	'Open' or 'Resolved'
created_at, resolved_by, resolved_at	TEXT	

Table 9: Table support_tickets

Table reports. Problems reported to the admin.

Column	Type	Constraint or meaning
id	INTEGER	PRIMARY KEY AUTOINCREMENT
reporter_email, reporter_role	TEXT	NOT NULL
category, offender_type, offender_email	TEXT	what happened and who it is about
subject, description	TEXT	NOT NULL
status, resolution	TEXT	'Open' or 'Resolved', resolution text
created_at, resolved_by, resolved_at	TEXT	

Table 10: Table reports

Table vehicle_types. Seats per vehicle type.

Column	Type	Constraint or meaning
id	INTEGER	PRIMARY KEY AUTOINCREMENT
name	TEXT	UNIQUE NOT NULL, for example 'desert buggy'
capacity	INTEGER	NOT NULL seats (2, 6, 20, 4, 50 by default)

Table 11: Table vehicle_types

Table vehicles. The dispatcher's fleet.

Column	Type	Constraint or meaning
id	INTEGER	PRIMARY KEY AUTOINCREMENT
license_number	TEXT	UNIQUE NOT NULL
vehicle_type, location	TEXT	NOT NULL
status	TEXT	'Available', 'Assigned' or 'Maintenance'
dispatcher_email	TEXT	owner dispatcher
auto_reassign_recurring	INTEGER	0 or 1

Table 12: Table vehicles

Table event_vehicles. Which vehicles serve which event.

Column	Type	Constraint or meaning
id	INTEGER	PRIMARY KEY AUTOINCREMENT
event_id, vehicle_id	INTEGER	FOREIGN KEYs events(id), vehicles(id)
status	TEXT	'Assigned' or 'Concluded'

Table 13: Table event_vehicles

Table partner_cart. A partner's shopping list.

Column	Type	Constraint or meaning
id	INTEGER	PRIMARY KEY AUTOINCREMENT
partner_email	TEXT	NOT NULL, indexed
event_id	INTEGER	NOT NULL, FOREIGN KEY events(id)
booking_date, guests, client_name	TEXT, INTEGER, TEXT	
created_at	TEXT	default CURRENT_TIMESTAMP

Table 14: Table partner_cart

Table vendor_categories. Categories a vendor can choose for events.

Column	Type	Constraint or meaning
id	INTEGER	PRIMARY KEY AUTOINCREMENT
vendor_email, name	TEXT	NOT NULL, no duplicate name per vendor

Table 15: Table vendor_categories

Indexes: idx_bookings_customer on bookings(customer_email), idx_bookings_event on bookings(event_id, booking_date), idx_cart_partner on partner_cart(partner_email) and idx_reviews_event on reviews(event_id).

3.0 Architectural and component-level design

Rihla uses a layered client and server architecture with three layers:

- **Presentation layer (client, src/):** Tkinter screens. navigate.py moves between the login, sign-up and password screens. screens/routing.py opens the home screen that belongs to the user's role. Every home screen extends BaseWelcomeScreen, which draws the side menu, the top bar with the clock, the Report, Settings and Log Out buttons, and a scrollable content area.
- **Service layer (server/server.py):** a Flask REST API. Each endpoint reads the JSON body, checks the caller with _blocked (unknown, banned or wrong role gives HTTP 403), calls one database function and returns JSON.
- **Data layer (database_core.py, database_events.py):** all SQL and all business rules: password hashing, SSN rules, capacity, VAT, ticket codes, refunds, venue scope and bans.

The client never trusts its own checks alone. For example, the event page checks the remaining seats before it opens the payment window, but the server checks them again inside a BEGIN IMMEDIATE transaction when the booking is saved. This is how double booking is prevented without the 15-minute seat lock that we planned in Phase 1.

3.1 System Structure

The client is organised by screen, and the server by data area. The client package welcome/ has one module per role plus shared views (ReportView, VehicleView, DispatchView, cancellation helpers and the shared table style). The server has one API module and two data modules. database_core.py handles users, SSN, staff venues, support tickets and reports. database_events.py handles venues, categories, events, capacity, bookings, VAT, cancellations, tickets, vehicles, dispatch, the partner cart and reviews.

3.1.1 Architecture diagram

Figure 3 shows the class hierarchy of the client screens. Figure 4 shows how the client modules, the Flask server and the data modules depend on each other. Python modules that are not classes are shown with the stereotype <<module>>.

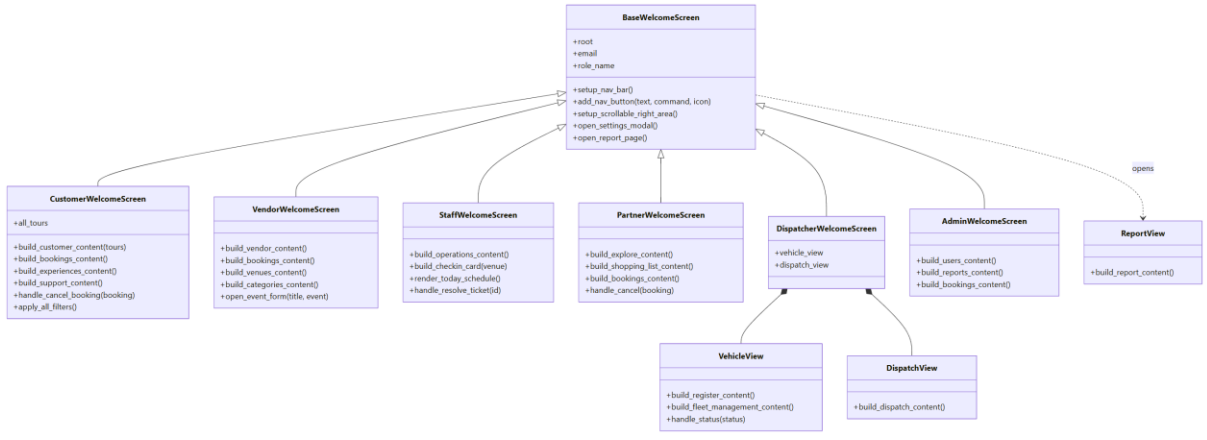


Figure 3: Design class diagram, part 1: client home screens and shared views

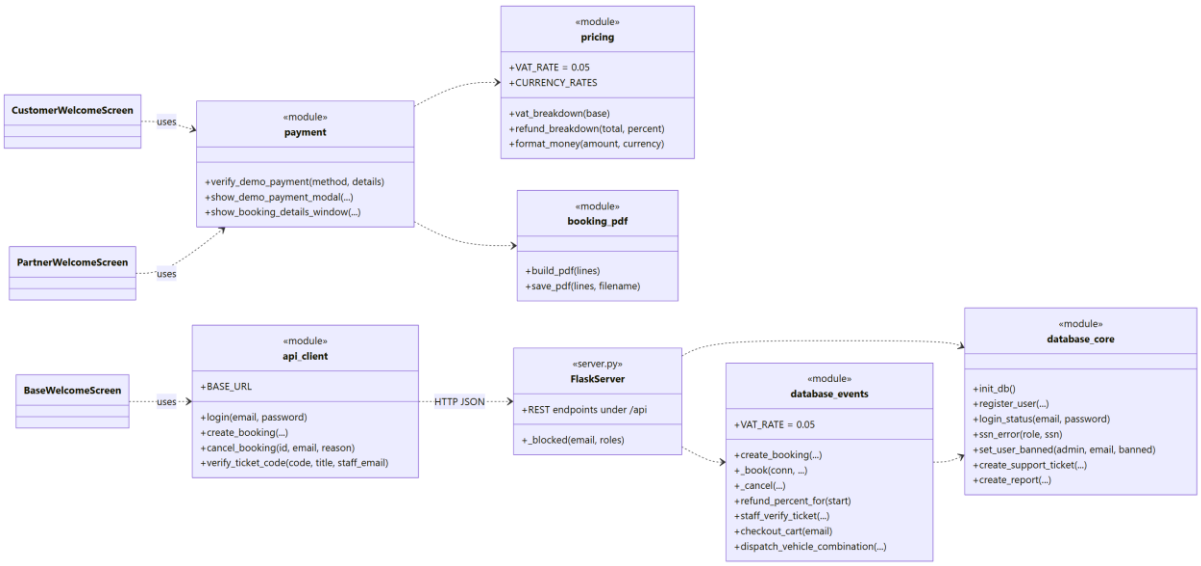


Figure 4: Design class diagram, part 2: client modules, Flask server and data modules

The use case diagrams in Figures 5, 6 and 7 show which functions each role can use. They are the base for the components described in section 3.2.

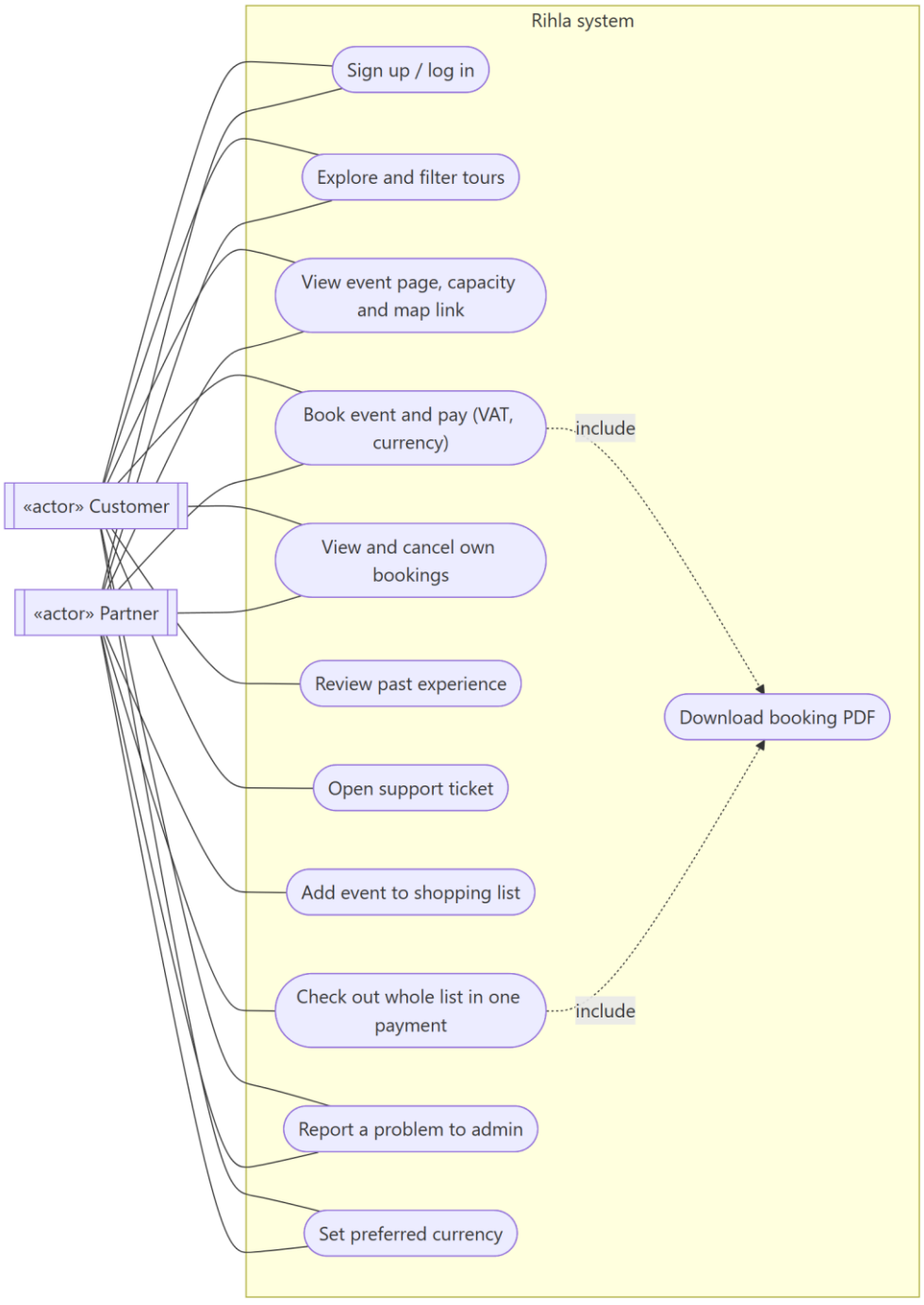


Figure 5: Use case diagram for Customer and Partner

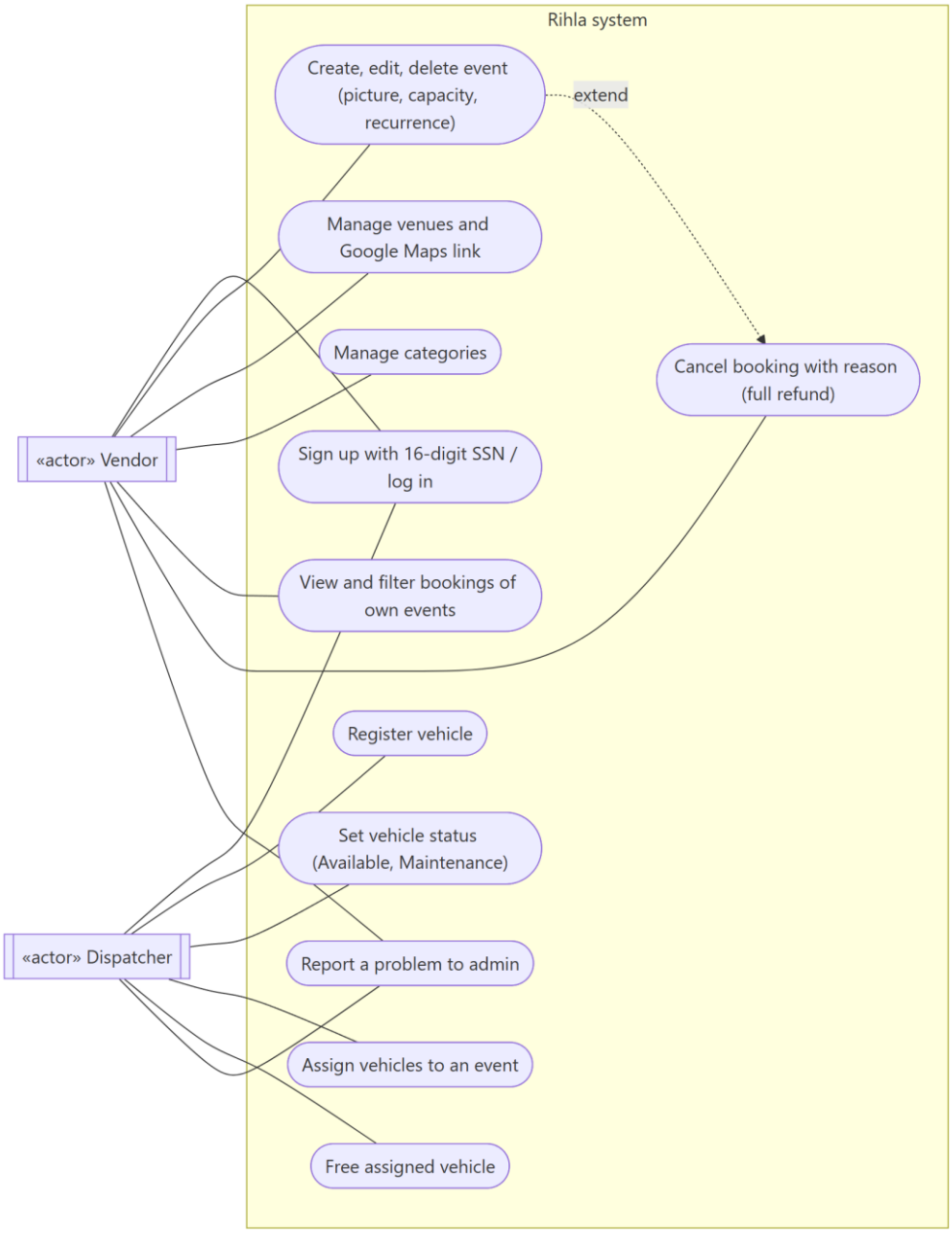


Figure 6: Use case diagram for Vendor and Dispatcher

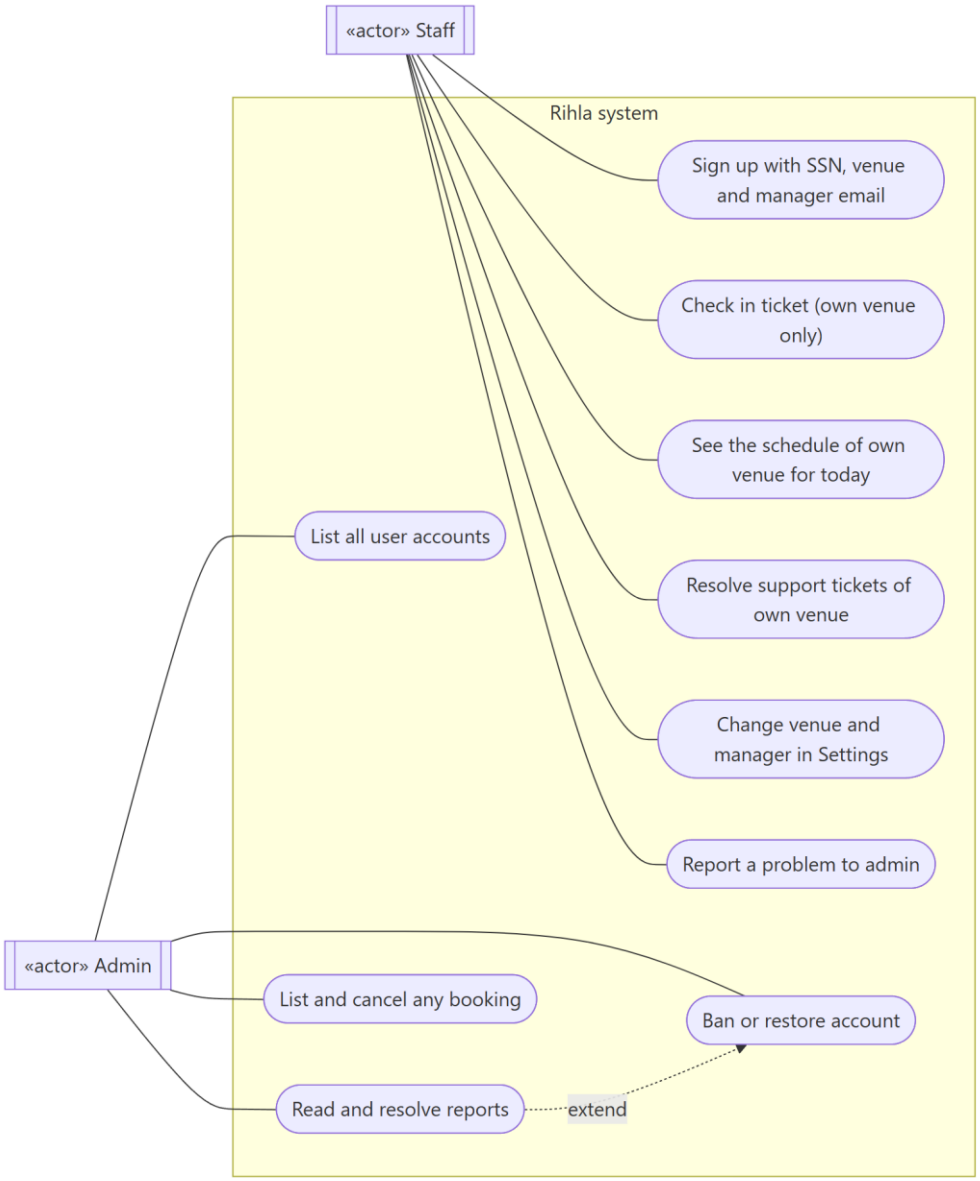


Figure 7: Use case diagram for Staff and Admin

3.2 Description of components (overview)

We describe eight components. For each one we give its responsibilities, its interface, its classes, its limits and the processing of its main operations. PDL means Program Design Language: short, structured pseudocode that follows the real code.

3.2 Description for Component 1: Authentication and Accounts

3.2.1 Processing narrative (PSPEC) for Authentication and Accounts

This component creates accounts and signs users in. It keeps passwords as salted PBKDF2 hashes with 200,000 rounds and upgrades any old plain-text password after a successful login. Sign-up has an account type picker with five roles. Admin is not in the list, so nobody can register as an admin. Every role except Customer must give a 16-digit SSN. Staff must also choose an existing venue and enter the email of the venue manager. The component also handles the forgot-password flow, the preferred currency and the login refusal for banned accounts.

3.2.2 Authentication and Accounts interface description

The client calls these REST endpoints through api_client.py. All bodies and answers are JSON.

Endpoint	Input	Output
POST /api/auth/login	email, password	email as stored and role, or 401 (wrong) or 403 (banned)
POST /api/users/register	email, password, role, phone, ssn, venue_id, manager_email	success, or a message explaining the problem
PUT /api/users/password	email, new_password	success
GET/PUT /api/users/currency	email, currency	currency code
GET/PUT /api/staff/venue	email, venue_id, manager_email	venue with manager email
GET /api/users/email_exists, phone_exists	email or phone	exists flag

Table 16: Interface of the Authentication and Accounts component

3.2.3 Authentication and Accounts processing detail

3.2.3.1 Design class hierarchy for Authentication and Accounts

Screens: login.show_login_screen, create_account.show_create_account_screen, forgot_password.show_forgot_password_screen, reset_password.show_reset_password_window, social_login.open_social_login and the DigitBoxes widget for the SSN. Server: database_core functions register_user, login_status, ssn_error, staff_venue_error, hash_password, verify_password, set_currency and set_staff_venue.

3.2.3.2 Restrictions and limitations for Authentication and Accounts

There is no session token. After login the client only remembers the email. Email and phone verification use the fixed code 000000. The Google and Apple buttons open a normal email and password form, not a real OAuth login. PUT /api/users/password does not check the old password.

3.2.3.3 Performance issues for Authentication and Accounts

PBKDF2 with 200,000 rounds takes a short but noticeable time on purpose. It runs once per login or sign-up, so it does not slow down normal use.

3.2.3.4 Design constraints for Authentication and Accounts

Passwords need at least 8 characters. The SSN must be exactly 16 digits (0 to 9) and not the same digit 16 times. The phone is a UAE number with 9 digits after +971.

3.2.3.5 Processing detail for each operation of Authentication and Accounts

3.2.3.5.1 Processing narrative (PSPEC) for each operation

1. login_status(email, password): Finds the user without letter case, verifies the password, upgrades an old plain-text password, and returns 'ok', 'invalid' or 'banned'.

2. register_user(email, password, role, phone, ssn, venue_id, manager_email): Saves a new account when the email is free and the role rules pass. Stores only the SSN hash and its last 4 digits.

3. ssn_error(role, ssn): Returns None when the SSN is fine for the role, otherwise the message to show.

3.2.3.5.2 Algorithmic model (PDL) for each operation

1. login_status(email, password)

```
FUNCTION login_status(email, password) RETURNS text
  row = SELECT email, password, banned FROM users WHERE lower(email) = lower(email)
  IF row is empty OR NOT verify_password(password, row.password) THEN RETURN 'invalid'
  IF row.password does not start with 'pbkdf2$' THEN
     UPDATE users SET password = hash_password(password)
  IF row.banned THEN RETURN 'banned'
  RETURN 'ok'
```

2. register_user(email, password, role, phone, ssn, venue_id, manager_email)

```
FUNCTION register_user(...) RETURNS boolean
  IF role NOT IN SIGNUP_ROLES THEN role = 'Customer'
  IF email empty OR password empty OR email_exists(email) OR ssn_error(role, ssn) THEN
RETURN false
  IF role = 'Staff' THEN
     IF staff_venue_error(venue_id, manager_email) THEN RETURN false
  ELSE venue_id = NULL, manager_email = NULL
  INSERT INTO users (email, hash_password(password), role, phone, now,
                     hash(ssn), last 4 digits of ssn, venue_id, manager_email)
  RETURN true
```

3. ssn_error(role, ssn)

```
FUNCTION ssn_error(role, ssn)
  IF role = 'Customer' THEN RETURN None
  ssn = ssn without spaces and dashes
  IF ssn empty THEN RETURN 'A 16-digit SSN is required'
  IF length(ssn) <> 16 OR ssn has a non-digit THEN RETURN 'must be exactly 16 digits'
  IF all digits are the same THEN RETURN 'Please enter your real SSN.'
  RETURN None
```

3.2 Description for Component 2: Event and Venue Management

3.2.1 Processing narrative (PSPEC) for Event and Venue Management

This component lets a vendor publish and maintain experiences. A vendor creates events with title, category, price in AED, description, venue, a one-time date or weekly days, a time, a capacity (1 to 5000, default 20) and an optional JPEG picture. The vendor also manages venues (with an optional Google Maps link) and categories. Renaming an event also renames it in the bookings and tickets. Deleting an event frees its vehicles, cancels its upcoming unused bookings with a full refund and deletes the picture.

3.2.2 Event and Venue Management interface description

The client calls these REST endpoints through api_client.py. All bodies and answers are JSON.

Endpoint	Input	Output
GET /api/events, /api/events/vendor/<email>	none or vendor email	list of events
POST /api/events	event fields, image_b64	success (Vendor role checked)
PUT /api/events/<id>	event fields, image_b64 or remove_image	success
DELETE /api/events/<id>	event id	success
GET /api/events/<id>/image	event id	JPEG bytes or 404
POST /api/venues, DELETE /api/venues/<id>	name, address, vendor_email, maps_url	success or 'in use'
PUT /api/venues/<id>/maps_url	vendor_email, maps_url	cleaned link or error
GET/POST/DELETE /api/vendor_categories	vendor_email, name	categories

Table 17: Interface of the Event and Venue Management component

3.2.3 Event and Venue Management processing detail

3.2.3.1 Design class hierarchy for Event and Venue Management

VendorWelcomeScreen (My Events, Bookings, Manage Venues, Manage Categories, Add New Event) with open_event_form for create and edit. event_images.py (load, encode, CoverImage). Server: create_event, update_event, delete_event, create_venue, delete_venue, validate_maps_url, update_venue_maps_url, add_vendor_category.

3.2.3.2 Restrictions and limitations for Event and Venue Management

Update and delete of events and delete of venues do not check that the caller owns them. A venue cannot be deleted while events use it or staff work there. Prices are stored as text and only the digits are used, so decimal prices are not supported.

3.2.3.3 Performance issues for Event and Venue Management

Pictures are shrunk to 1400 pixels on the client before upload and limited to 4 MB on the server, so lists with many pictures still load quickly.

3.2.3.4 Design constraints for Event and Venue Management

Only https links on Google Maps hosts are accepted for venues. Pictures must be JPEG.

3.2.3.5 Processing detail for each operation of Event and Venue Management

3.2.3.5.1 Processing narrative (PSPEC) for each operation

1. delete_event(event_id): Frees vehicles, cancels upcoming bookings with 100% refund, cancels their tickets and deletes the event.

2. validate_maps_url(url): Accepts an empty link or an https Google Maps link and returns the cleaned link or an error message.

3.2.3.5.2 Algorithmic model (PDL) for each operation

1. delete_event(event_id)

```
PROCEDURE delete_event(event_id)
  vendor = owner of event
  UPDATE vehicles SET status = 'Available' WHERE assigned to this event
  DELETE FROM event_vehicles WHERE event_id = event_id
  UPDATE bookings SET status = 'Cancelled', cancelled_by = vendor, cancelled_by_role =
'Vendor',
         cancel_reason = 'The vendor removed this event.', refund_percent = 100,
         refund_amount = total_price
     WHERE event_id = event_id AND status = 'Confirmed' AND booking_date >= today
       AND ticket is still 'Valid'
  UPDATE tickets SET status = 'Cancelled' for those bookings
  DELETE FROM events WHERE id = event_id
```

2. validate_maps_url(url)

```
FUNCTION validate_maps_url(url) RETURNS (clean_url, error)
  url = trimmed url
  IF url empty THEN RETURN ('', None)
  IF length > 500 OR url contains spaces THEN RETURN error
  add 'https://' when there is no scheme, change 'http://' to 'https://'
  IF host is maps.app.goo.gl OR goo.gl/maps OR a google domain with path /maps THEN RETURN
(url, None)
  RETURN ('', 'Please use a Google Maps link')
```

3.2 Description for Component 3: Booking, Pricing and Payment

3.2.1 Processing narrative (PSPEC) for Booking, Pricing and Payment

This component turns a choice of event, date and guests into a paid booking. The event page shows a calendar with only the allowed dates, the capacity bar ('N of TOTAL remaining', 'Hurry up!' when few seats are left) and the price. The payment window shows the base price, VAT (5%) and total in the currency the user picks. After a demo payment is approved the server saves the booking and a new ticket. The Booking Details window then shows every detail and can save a PDF. Customers book 1 to 10 guests. Partners have no per-booking guest limit and can collect several events in a shopping list and pay for all of them at once.

3.2.2 Booking, Pricing and Payment interface description

The client calls these REST endpoints through api_client.py. All bodies and answers are JSON.

Endpoint	Input	Output
GET /api/events/<id>/capacity?date=	event id, date	total, source, vehicles, booked, remaining
POST /api/bookings	customer_email, event_id, booking_date, guests, client_name	booking with base_price, vat, total_price, ticket_code
GET /api/bookings/customer/<email>	email	list of bookings with status and refund info
POST /api/cart, GET /api/cart/<email>	partner_email, event_id, booking_date, guests, client_name	success, cart items
POST /api/cart/<email>/checkout	partner email	list of bookings, or nothing booked

Table 18: Interface of the Booking, Pricing and Payment component

3.2.3 Booking, Pricing and Payment processing detail

3.2.3.1 Design class hierarchy for Booking, Pricing and Payment

screens/event_screen.show_event_screen, CustomerWelcomeScreen, PartnerWelcomeScreen, payment.py (verify_demo_payment, show_demo_payment_modal, show_booking_details_window), pricing.py (vat_breakdown, convert, format_money), booking_pdf.py (build_pdf, save_pdf) and the CapacityBar widget. Server: create_booking, _book, _capacity_info, add_vat, _new_ticket_code, add_to_cart, checkout_cart.

3.2.3.2 Restrictions and limitations for Booking, Pricing and Payment

Payment is a demo. It only checks that the form still holds the fixed demo card or bank details. Currency rates are fixed numbers, not live rates, and every amount is charged in AED. The payment is approved on the client before the server saves the booking, so if the booking then fails the Booking Details window says 'NOT CONFIRMED' and asks the user to contact support.

3.2.3.3 Performance issues for Booking, Pricing and Payment

A booking is one short write transaction. BEGIN IMMEDIATE takes the write lock at the start, so two bookings for the last seats are handled one after the other.

3.2.3.4 Design constraints for Booking, Pricing and Payment

The date must be today or later and allowed by the event (the right weekday for recurring events, the exact date for one-time events). Guests must not be more than the seats left on that date.

3.2.3.5 Processing detail for each operation of Booking, Pricing and Payment

3.2.3.5.1 Processing narrative (PSPEC) for each operation

1. _book(conn, email, event_id, date, guests, is_partner, client_name): Validates the request, checks capacity for that date, adds VAT, creates a unique 16-digit ticket and saves the booking, all inside the caller's transaction.

2. _capacity_info(conn, event_id, day): Total seats come from the assigned vehicles when there are any, otherwise from the event capacity. Booked seats are the confirmed guests on that date.

3. checkout_cart(partner_email): Books every cart item in one transaction. If one item fails, none is kept and the cart stays as it was.

4. vat_breakdown(base) and format_money(amount, currency): Client helpers used by the payment window and the receipt.

3.2.3.5.2 Algorithmic model (PDL) for each operation

1. _book(conn, email, event_id, date, guests, is_partner, client_name)

```
FUNCTION _book(conn, email, event_id, date, guests, is_partner, client_name)
  IF date or guests invalid THEN REFUSE 'Invalid date or number of guests.'
  IF guests < 1 THEN REFUSE
  IF NOT is_partner AND guests > 10 THEN REFUSE 'You can book 1 to 10 guests at a time.'
  IF date < today THEN REFUSE
  event = SELECT event with venue
  IF event missing OR NOT _event_allows_date(event, date) THEN REFUSE
  cap = _capacity_info(conn, event_id, date)
  IF guests > cap.remaining THEN REFUSE 'fully booked' or 'Not enough seats'
  code = _new_ticket_code(conn)        -- random 16 digits not used before
  (base, vat, total) = add_vat(price_to_int(event.price) * guests)
  INSERT INTO tickets (code, title, email, 'Valid')
  INSERT INTO bookings (..., total, code, 'Confirmed', client_name)
  RETURN success with booking, base and vat
```

2. _capacity_info(conn, event_id, day)

```
FUNCTION _capacity_info(conn, event_id, day)
  groups = assigned vehicles of the event grouped by type, with seats per type
  IF groups not empty THEN total = SUM(count * seats_per_type), source = 'vehicles'
  ELSE total = events.capacity (or 20), source = 'event'
  booked = SUM(guests) of 'Confirmed' bookings for event_id on day
  remaining = MAX(0, total - booked)
  RETURN total, source, booked, remaining
```

3. checkout_cart(partner_email)

```
FUNCTION checkout_cart(partner_email)
  items = get_cart_items(partner_email)
  IF items empty THEN REFUSE 'Shopping list is empty.'
  BEGIN IMMEDIATE
  FOR EACH item IN items
     result = _book(conn, partner_email, item..., is_partner = true)
     IF NOT result.success THEN ROLLBACK and REFUSE 'No booking was made'
  COMMIT
  clear_cart(partner_email)
  RETURN all bookings
```

4. vat_breakdown(base) and format_money(amount, currency)

```
FUNCTION vat_breakdown(base)
  base = round(base, 2), vat = round(base * 0.05, 2)
  RETURN (base, vat, round(base + vat, 2))
FUNCTION format_money(amount_aed, currency)
  RETURN round(amount_aed * CURRENCY_RATES[currency], 2) formatted with the currency code
```

3.2 Description for Component 4: Cancellation and Refund

3.2.1 Processing narrative (PSPEC) for Cancellation and Refund

This component cancels bookings under one policy. A customer or partner can cancel their own booking and gets 100% back 48 hours or more before the start, 50% between 48 and 24 hours, and cannot cancel in the last 24 hours. A vendor (for bookings on their own events) and an admin (for any booking) can cancel until the event starts, always with a full refund, and must give a reason of at most 300 characters. VAT is refunded in the same proportion. Every cancellation stores when, who, which role, why, the percent and the amount. Cancelling frees the seats at once.

3.2.2 Cancellation and Refund interface description

The client calls these REST endpoints through api_client.py. All bodies and answers are JSON.

Endpoint	Input	Output
POST /api/bookings/<id>/cancel	customer_email, reason	refund percent, base, vat, total
POST /api/bookings/<id>/vendor_cancel	vendor_email, reason	refund (Vendor role checked)
POST /api/admin/bookings/<id>/cancel	admin_email, reason	refund (admin checked)
GET /api/admin/bookings	admin_email	every booking

Table 19: Interface of the Cancellation and Refund component

3.2.3 Cancellation and Refund processing detail

3.2.3.1 Design class hierarchy for Cancellation and Refund

welcome/cancellation.py (policy_line, cancelled_line, confirm_own_cancel, ask_reason), pricing.refund_breakdown and the cancel buttons in the customer, partner, vendor and admin screens. Server: _cancel, cancel_booking, vendor_cancel_booking, admin_cancel_booking, refund_percent_for, split_refund, _event_start.

3.2.3.2 Restrictions and limitations for Cancellation and Refund

Refunds are recorded only. No money is sent anywhere. A booking without a time counts from midnight of its date.

3.2.3.3 Performance issues for Cancellation and Refund

One short transaction per cancellation (BEGIN IMMEDIATE).

3.2.3.4 Design constraints for Cancellation and Refund

A used (checked-in) ticket cannot be cancelled. A started event cannot be cancelled by anyone.

3.2.3.5 Processing detail for each operation of Cancellation and Refund

3.2.3.5.1 Processing narrative (PSPEC) for each operation

1. _cancel(booking_id, actor_email, actor_role, reason, owner_check, by_operator): Shared logic for all three kinds of cancellation. Figure 14 shows the same decision as an activity diagram.

2. refund_percent_for(start) and split_refund(total, percent): The policy steps and the VAT split of a refund.

3.2.3.5.2 Algorithmic model (PDL) for each operation

1. _cancel(booking_id, actor_email, actor_role, reason, owner_check, by_operator)

```
FUNCTION _cancel(booking_id, actor, role, reason, owner_check, by_operator)
  IF length(reason) > 300 THEN REFUSE
  IF by_operator AND reason empty THEN REFUSE 'Please give the customer a reason'
  BEGIN IMMEDIATE
  row = booking joined with its event
  IF row missing OR owner_check(row) fails THEN REFUSE
  IF row.status <> 'Confirmed' THEN REFUSE
  start = _event_start(row.date, row.time)
  IF now >= start THEN REFUSE 'already started'
  percent = 100 IF by_operator ELSE refund_percent_for(start)
  IF percent is None THEN REFUSE 'Cancellations close 24 hours before'
  IF UPDATE tickets SET status = 'Cancelled' WHERE code AND status = 'Valid' changed no
row
     THEN REFUSE 'ticket was already used'
  (base, vat, refund) = split_refund(row.total_price, percent)
  UPDATE bookings SET status = 'Cancelled', cancelled_at = now, cancelled_by = actor,
         cancelled_by_role = role, cancel_reason = reason, refund_percent = percent,
         refund_amount = refund
  RETURN success with refund
```

2. refund_percent_for(start) and split_refund(total, percent)

```
FUNCTION refund_percent_for(start)
  hours_left = (start - now) in hours
  IF hours_left >= 48 THEN RETURN 100
  IF hours_left >= 24 THEN RETURN 50
  RETURN None
FUNCTION split_refund(total, percent)
  refund = round(total * percent / 100, 2)
  base = round(refund / 1.05, 2)
  RETURN (base, refund - base, refund)
```

3.2 Description for Component 5: Venue Operations (Staff)

3.2.1 Processing narrative (PSPEC) for Venue Operations (Staff)

Every staff account works at one venue. The Venue Operations page shows the venue, its manager email, a ticket check-in card and the schedule of the day for that venue (bookings, guests and guests already checked in). The server enforces the venue scope: a ticket for another venue is refused, the schedule never includes other venues, and the Support Tickets page lists only questions about the staff member's venue (old tickets that name no event are shown to all staff). A ticket can be checked in only once.

3.2.2 Venue Operations (Staff) interface description

The client calls these REST endpoints through api_client.py. All bodies and answers are JSON.

Endpoint	Input	Output
POST /api/tickets/verify	staff_email, ticket_code, expected_event_title	access granted with guest count, or refusal
GET /api/events/today?staff_email=	staff email	today's events of the venue, or 403
GET /api/support/open?staff_email=	staff email	open tickets of the venue, or 403
POST /api/support/<id>/resolve	staff_email	resolved, or not for your venue

Table 20: Interface of the Venue Operations (Staff) component

3.2.3 Venue Operations (Staff) processing detail

3.2.3.1 Design class hierarchy for Venue Operations (Staff)

StaffWelcomeScreen (build_operations_content, build_checkin_card, render_today_schedule, build_support_content), DigitBoxes for the 16-digit code. Server: staff_verify_ticket, verify_ticket_code, _ticket_venue_id, get_staff_schedule, get_events_today, get_staff_support_tickets, staff_resolve_support_ticket.

3.2.3.2 Restrictions and limitations for Venue Operations (Staff)

Check-in uses a typed code. There is no QR scanner. Staff see only the schedule of today.

3.2.3.3 Performance issues for Venue Operations (Staff)

get_events_today runs two small queries per event of the venue, which is fine for the number of events one venue has in a day.

3.2.3.4 Design constraints for Venue Operations (Staff)

The update to 'Used' has the condition status = 'Valid', so two staff members cannot admit the same ticket twice even at the same moment.

3.2.3.5 Processing detail for each operation of Venue Operations (Staff)

3.2.3.5.1 Processing narrative (PSPEC) for each operation

1. staff_verify_ticket(staff_email, code, expected_title): Finds the staff venue and checks the ticket in only if it belongs to that venue.

3.2.3.5.2 Algorithmic model (PDL) for each operation

1. staff_verify_ticket(staff_email, code, expected_title)

```
FUNCTION staff_verify_ticket(staff_email, code, expected_title)
  venue = get_staff_venue(staff_email)     -- role Staff, not banned
  IF venue is None THEN REFUSE 'Only staff assigned to a venue can check tickets in.'
  ticket = SELECT event_title, customer_email, status FROM tickets WHERE ticket_code =
code
  IF ticket missing THEN REFUSE 'Invalid ticket code.'
  IF venue of the ticket's event <> venue.id THEN REFUSE 'not for an event at your venue'
  IF expected_title given AND differs THEN REFUSE 'Wrong event.'
  IF status = 'Cancelled' OR status = 'Used' THEN REFUSE
  UPDATE tickets SET status = 'Used' WHERE ticket_code = code AND status = 'Valid'
  IF no row changed THEN REFUSE 'just used or cancelled'
```

```
RETURN 'Access granted' with the number of guests and a note if the date is not today
```

3.2 Description for Component 6: Fleet and Dispatch

3.2.1 Processing narrative (PSPEC) for Fleet and Dispatch

The dispatcher registers vehicles (license number, type, location), keeps their status (Available, Assigned, Maintenance) and assigns a combination of available vehicles to an event. When an event has assigned vehicles, its capacity becomes the sum of their seats (for example two 4x4 Land Cruisers give 12 seats). Freeing a vehicle or setting it Available or Maintenance ends its assignment, and the event capacity goes back to the vendor's number.

3.2.2 Fleet and Dispatch interface description

The client calls these REST endpoints through api_client.py. All bodies and answers are JSON.

Endpoint	Input	Output
GET /api/vehicle_types	none	types with seats
GET /api/vehicles/<dispatcher_email>	email	vehicles of that dispatcher
POST /api/vehicles, DELETE /api/vehicles/<id>	vehicle fields	success
PUT /api/vehicles/<id>/status, POST /api/vehicles/<id>/free	status	success
POST /api/dispatch	event_id, vehicle_assignments	number assigned
GET /api/dispatch/<event_id>/vehicles	event id	assigned vehicles

Table 21: Interface of the Fleet and Dispatch component

3.2.3 Fleet and Dispatch processing detail

3.2.3.1 Design class hierarchy for Fleet and Dispatch

DispatcherWelcomeScreen with VehicleView (Register Vehicles, Fleet Maintenance) and DispatchView (Event Dispatch). Both use table_style.py for filterable tables. Server: create_vehicle, update_vehicle_status, free_vehicle, dispatch_vehicle_combination, get_dispatches_by_event.

3.2.3.2 Restrictions and limitations for Fleet and Dispatch

The vehicle and dispatch endpoints do not check the caller's role or ownership. Vehicles are assigned to an event as a whole, not to a single date. The auto_reassign_recurring flag is stored and shown but no scheduler uses it yet.

3.2.3.3 Performance issues for Fleet and Dispatch

Capacity is computed with one grouped query, so it stays fast with many vehicles.

3.2.3.4 Design constraints for Fleet and Dispatch

Only vehicles with status 'Available' can be assigned. The UPDATE has this condition, so a vehicle cannot be assigned twice.

3.2.3.5 Processing detail for each operation of Fleet and Dispatch

3.2.3.5.1 Processing narrative (PSPEC) for each operation

1. dispatch_vehicle_combination(event_id, assignments): Assigns each available vehicle and records the link.

2. update_vehicle_status(vehicle_id, status): Ends the assignment when the vehicle leaves 'Assigned'.

3.2.3.5.2 Algorithmic model (PDL) for each operation

1. dispatch_vehicle_combination(event_id, assignments)

```
FUNCTION dispatch_vehicle_combination(event_id, assignments)
  assigned = 0
  FOR EACH item IN assignments
     changed = UPDATE vehicles SET status = 'Assigned', auto_reassign_recurring =
item.flag
                WHERE id = item.vehicle_id AND status = 'Available'
     IF changed THEN
        INSERT INTO event_vehicles (event_id, item.vehicle_id, 'Assigned')
        assigned = assigned + 1
  RETURN assigned
```

2. update_vehicle_status(vehicle_id, status)

```
PROCEDURE update_vehicle_status(vehicle_id, status)
  IF status <> 'Assigned' THEN
     UPDATE event_vehicles SET status = 'Concluded' WHERE vehicle_id AND status =
'Assigned'
  UPDATE vehicles SET status = status WHERE id = vehicle_id
```

3.2 Description for Component 7: Administration and Reports

3.2.1 Processing narrative (PSPEC) for Administration and Reports

The admin keeps the platform safe. The User Accounts page lists every account with role, sign-up date, account age, status and, for staff, venue and manager email. The admin can ban and restore accounts, but not their own account or another admin. A banned user cannot log in (HTTP 403) and every write request from that email is refused by _blocked. Every other role has a Report button. A report has a role-specific category, who it is about, an optional offender email, a summary and details. The admin reads open reports first, resolves them or bans the offender (which also resolves the report). The admin also sees and can cancel every booking.

3.2.2 Administration and Reports interface description

The client calls these REST endpoints through api_client.py. All bodies and answers are JSON.

Endpoint	Input	Output
GET /api/admin/users?admin_email=	admin email	all accounts, or 403
POST /api/admin/users/ban	admin_email, email, banned	success or reason

Endpoint	Input	Output
POST /api/reports	reporter_email, category, offender_type, offender_email, subject, description	report id
GET /api/admin/reports	admin_email	reports, open first
POST /api/admin/reports/<id>/resolve	admin_email, resolution	success

Table 22: Interface of the Administration and Reports component

3.2.3 Administration and Reports processing detail

3.2.3.1 Design class hierarchy for Administration and Reports

AdminWelcomeScreen (build_users_content, build_reports_content, build_bookings_content, _open_ban_picker), ReportView with REPORT_OPTIONS per role. Server: is_admin, active_role, get_all_users, set_user_banned, create_report, get_reports, resolve_report, _blocked.

3.2.3.2 Restrictions and limitations for Administration and Reports

Admin requests are trusted by the admin_email they carry. The server checks that this email belongs to an active admin, but there is no token that proves who sent the request.

3.2.3.3 Performance issues for Administration and Reports

The user and report lists are read in one query each.

3.2.3.4 Design constraints for Administration and Reports

A user can have at most 5 open reports and cannot report their own account. Admins cannot file reports.

3.2.3.5 Processing detail for each operation of Administration and Reports

3.2.3.5.1 Processing narrative (PSPEC) for each operation

1. set_user_banned(admin_email, email, banned): Bans or restores one account.

2. _blocked(email, *roles): Guard used by the write endpoints of server.py.

3.2.3.5.2 Algorithmic model (PDL) for each operation

1. set_user_banned(admin_email, email, banned)

```
FUNCTION set_user_banned(admin_email, email, banned)
  IF NOT is_admin(admin_email) THEN REFUSE 'Only an admin can do this.'
  IF email = admin_email THEN REFUSE 'You cannot ban your own account.'
  row = role of email
  IF row missing THEN REFUSE
  IF row.role = 'Admin' THEN REFUSE 'Admin accounts cannot be banned.'
  IF banned THEN UPDATE users SET banned = 1, banned_at = now, banned_by = admin_email
  ELSE UPDATE users SET banned = 0, banned_at = NULL, banned_by = NULL
```

2. _blocked(email, *roles)

```
FUNCTION _blocked(email, roles)
  role = active_role(email)        -- None when unknown or banned
  IF role is None THEN RETURN HTTP 403 'unknown or banned'
```

```
  IF roles given AND role NOT IN roles THEN RETURN HTTP 403 'Only <roles> accounts can do
this.'
  RETURN None
```

3.2 Description for Component 8: Reviews and Support Tickets

3.2.1 Processing narrative (PSPEC) for Reviews and Support Tickets

After an experience a customer opens My Experiences, which lists bookings that were checked in or whose date has passed. The customer gives a rating from 0 to 5 in steps of 0.5 and an optional comment. A second review of the same booking updates the first one. The explore page shows the average rating per event, rounded to one decimal. Customers can also open support tickets about an event, and venue staff answer them (see the Venue Operations component).

3.2.2 Reviews and Support Tickets interface description

The client calls these REST endpoints through api_client.py. All bodies and answers are JSON.

Endpoint	Input	Output
GET /api/experiences/customer/<email>	email	past bookings with their review
POST /api/reviews	customer_email, booking_id, rating, comment	thank you message
GET /api/events/ratings	none	average and count per event
POST /api/support	customer_email, event_title, subject, message	ticket id
GET /api/support/customer/<email>	email	the customer's tickets

Table 23: Interface of the Reviews and Support Tickets component

3.2.3 Reviews and Support Tickets processing detail

3.2.3.1 Design class hierarchy for Reviews and Support Tickets

CustomerWelcomeScreen (build_experiences_content, open_review_window, build_support_content). Server: get_my_experiences, submit_review, get_event_ratings, _ensure_reviews_table, create_support_ticket.

3.2.3.2 Restrictions and limitations for Reviews and Support Tickets

Staff can mark a ticket resolved but cannot write an answer inside the app.

3.2.3.3 Performance issues for Reviews and Support Tickets

Ratings for all events are computed with one GROUP BY query.

3.2.3.4 Design constraints for Reviews and Support Tickets

A support ticket must name an event that is listed or that the customer booked before. Subject needs 3 characters, message 10 characters, and a customer can have at most 3 open tickets.

3.2.3.5 Processing detail for each operation of Reviews and Support Tickets

3.2.3.5.1 Processing narrative (PSPEC) for each operation

1. submit_review(email, booking_id, rating, comment): Saves or updates a review of an attended booking.

3.2.3.5.2 Algorithmic model (PDL) for each operation

1. submit_review(email, booking_id, rating, comment)

```
FUNCTION submit_review(email, booking_id, rating, comment)
  IF rating < 0 OR rating > 5 OR rating * 2 is not whole THEN REFUSE
  IF length(comment) > 500 THEN REFUSE
  allowed = bookings of email with status 'Checked in' or 'Expired'
  IF booking_id NOT IN allowed THEN REFUSE 'only experiences that you already attended'
  INSERT INTO reviews ... ON CONFLICT(booking_id) DO UPDATE rating, comment, updated_at
```

3.3 Dynamic behavior of the components

This section shows how the classes and modules work together at run time. Each sequence diagram follows the real function calls in the code. The state and activity diagrams show the life of a booking, the life of a vehicle and the refund decision.

3.3.1 Interaction diagrams

Booking with VAT and currency (component 3). The capacity is read first for the chosen date. The payment window adds VAT and converts the display currency. Only after the demo payment is approved does the client send the booking, and the server checks capacity again inside its transaction.

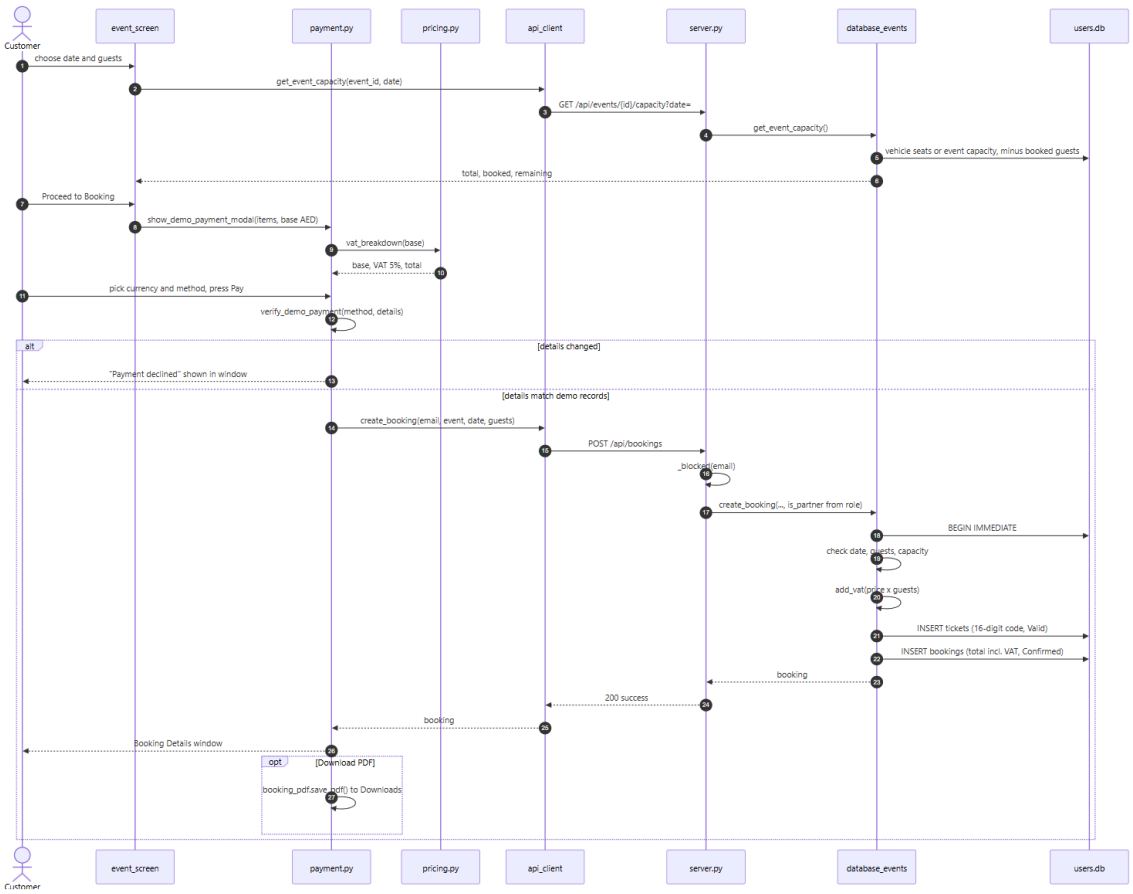


Figure 8: Sequence diagram: customer books an event and pays

Cancellation with refund (component 4). The client shows the refund the user would get before they confirm. The server decides the real percent and marks the ticket as cancelled only if it is still valid.

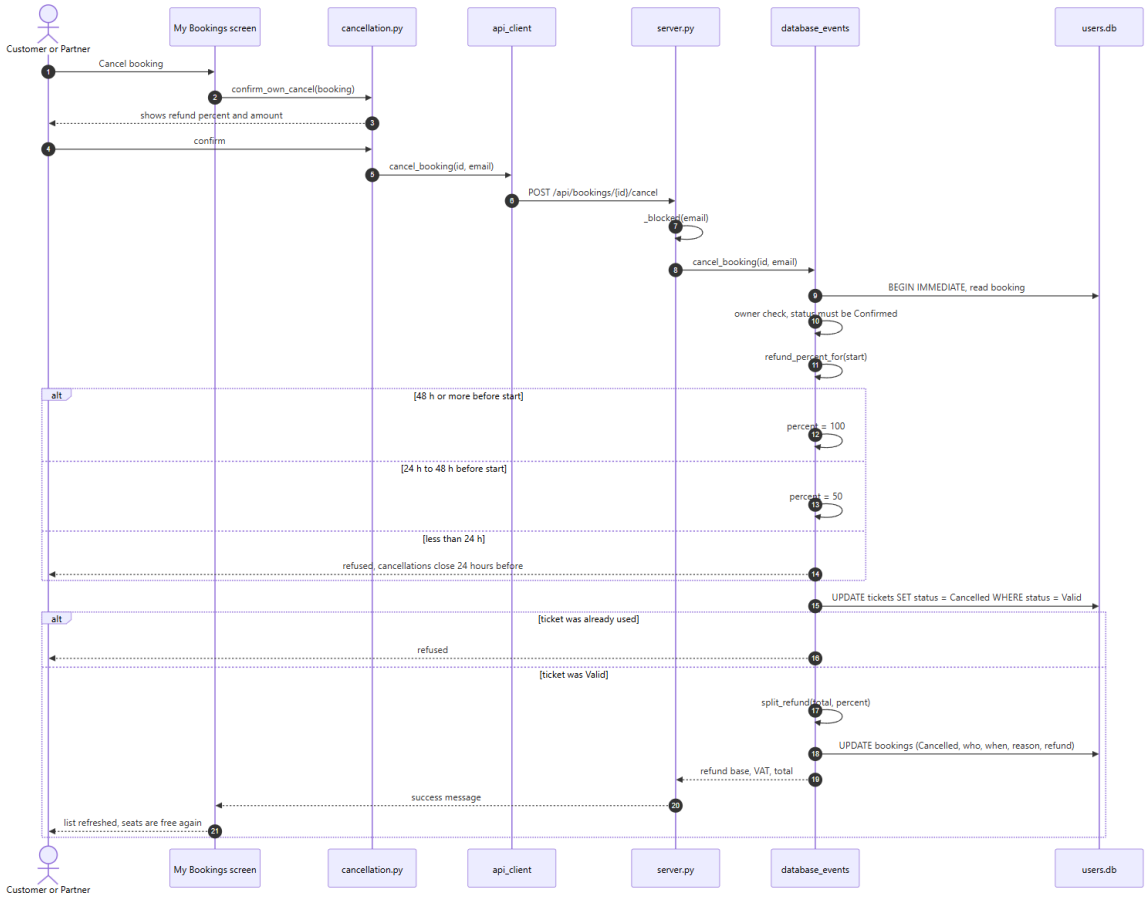


Figure 9: Sequence diagram: booker cancels a booking and gets a refund

Ticket check-in (component 5). The venue of the staff member is found on the server. A ticket for another venue is refused without giving details about it.

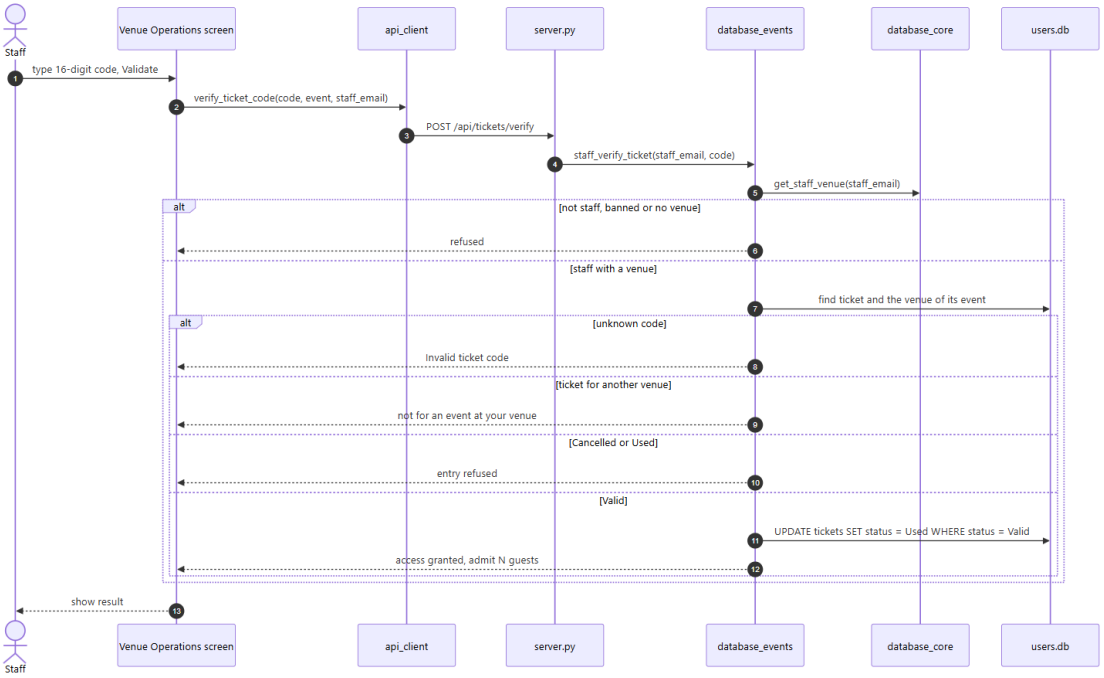


Figure 10: Sequence diagram: staff checks a ticket in

Partner checkout (component 3). All cart items are booked inside one transaction, so the partner never gets half a cart.

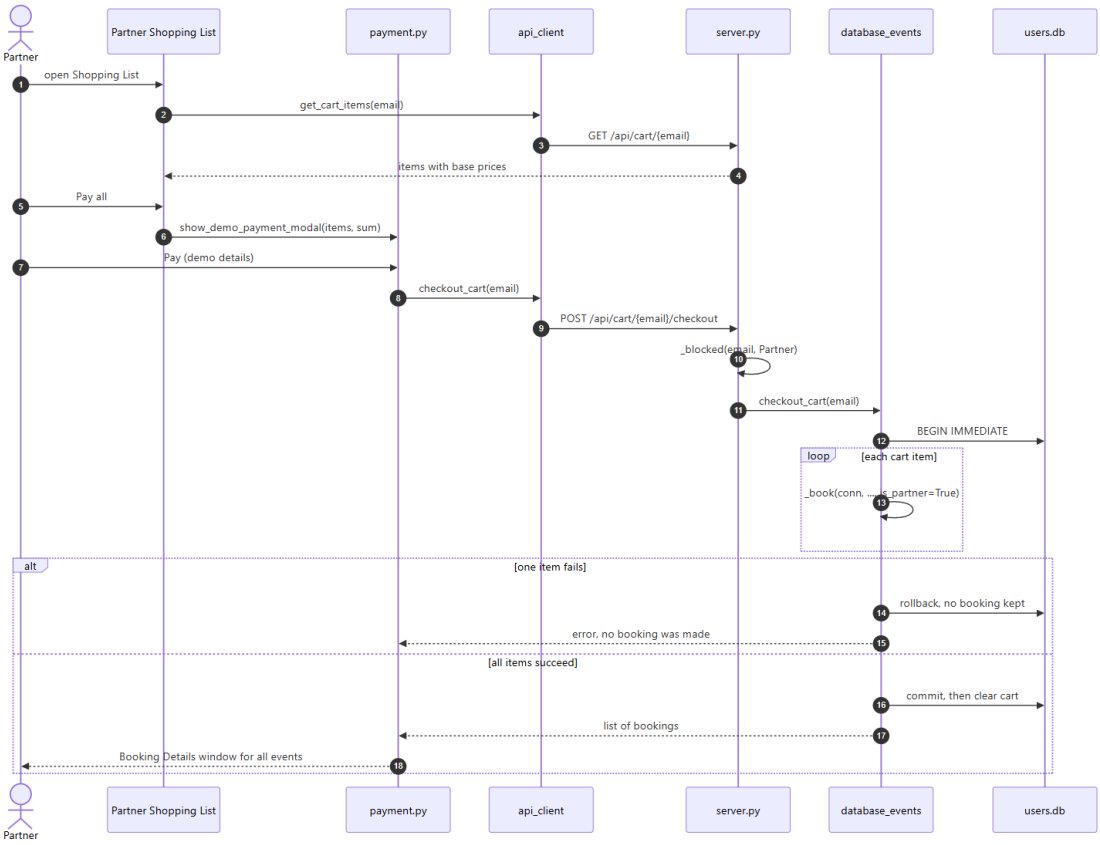


Figure 11: Sequence diagram: partner pays the whole shopping list at once

Report and ban (component 7). A report reaches the admin, who bans the offender and resolves the report.

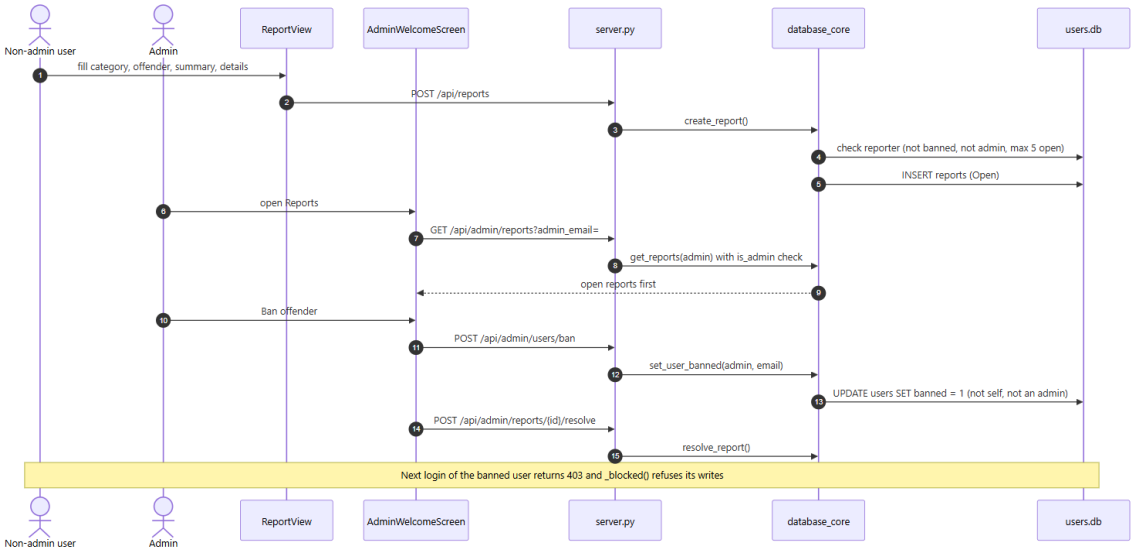


Figure 12: Sequence diagram: report and ban

3.3.2 State and activity diagrams

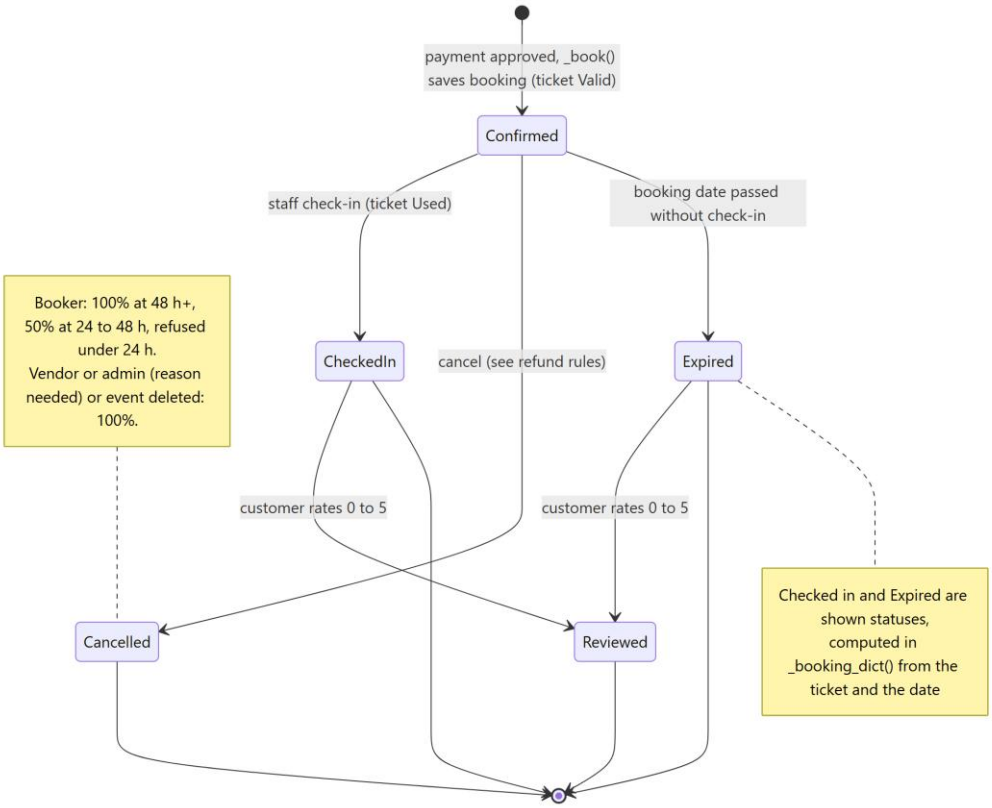


Figure 13: State diagram of a booking and its ticket

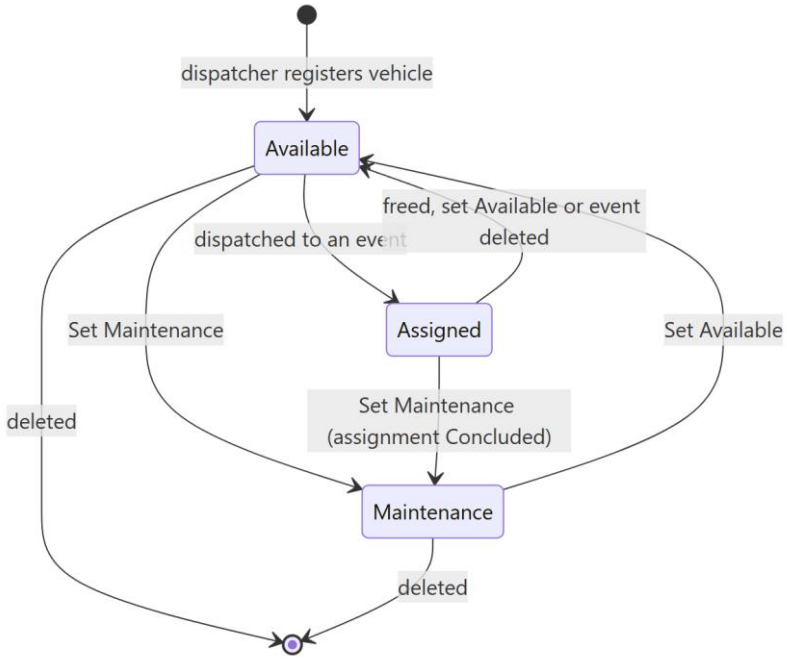


Figure 14: State diagram of a vehicle

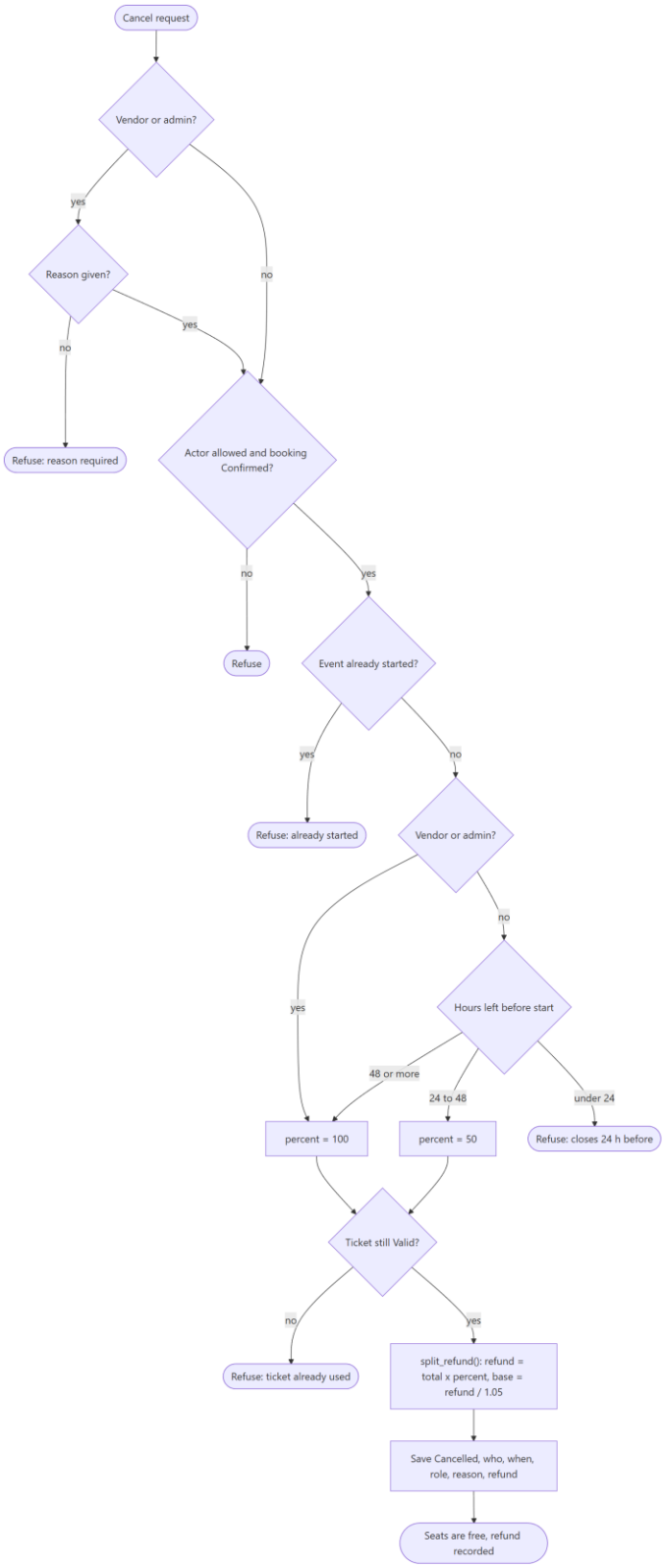


Figure 15: Activity diagram of the refund decision in _cancel()

4.0 User interface design

Rihla is a desktop program that opens maximised. The login, sign-up and password screens use a split layout: a picture of Dubai with the Rihla logo on the left and the form on the right. After login every role gets the same frame: a dark side menu on the left with the role name, the role's pages, and the Report, Settings and Log Out buttons at the bottom. A white top bar shows who is signed in and a live clock. The main area scrolls.

4.1 Description of the user interface

4.1.1 Screen images

The screenshots below were taken from the running application with the demo data from dummy_data.ps1.

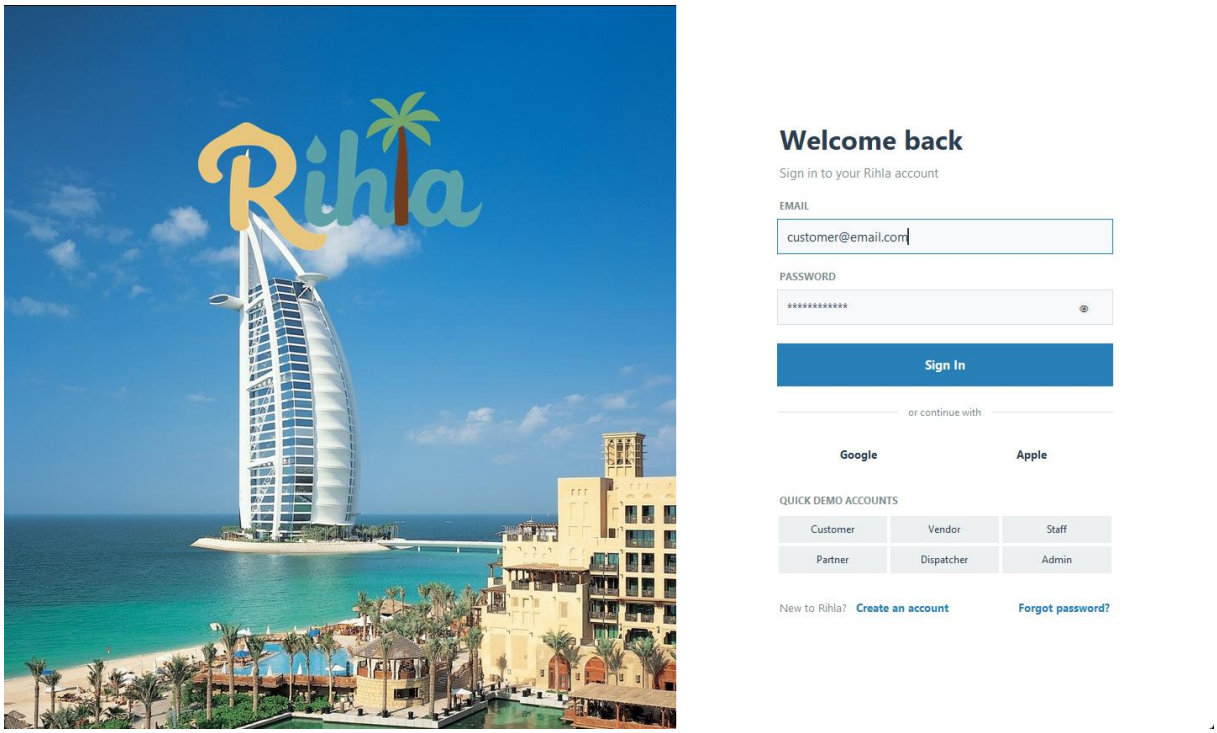


Figure 16: Login screen with Google and Apple buttons and quick demo account chips

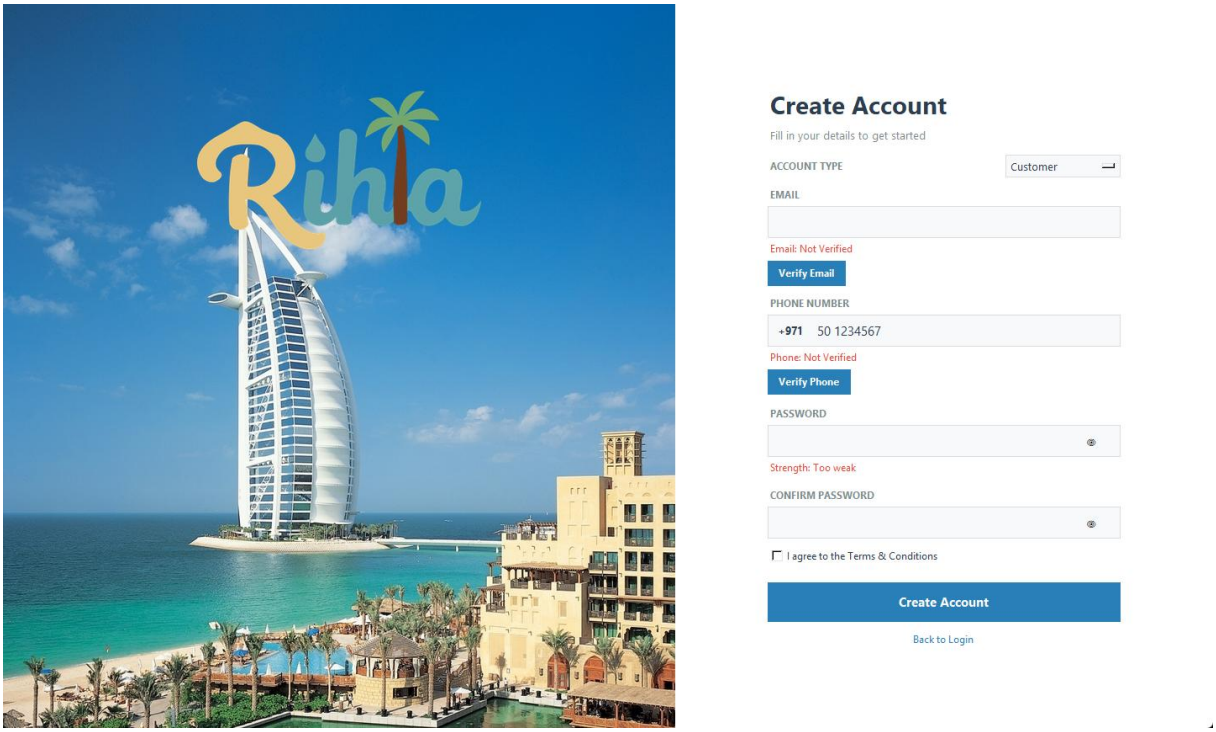


Figure 17: Create Account screen with the account type picker

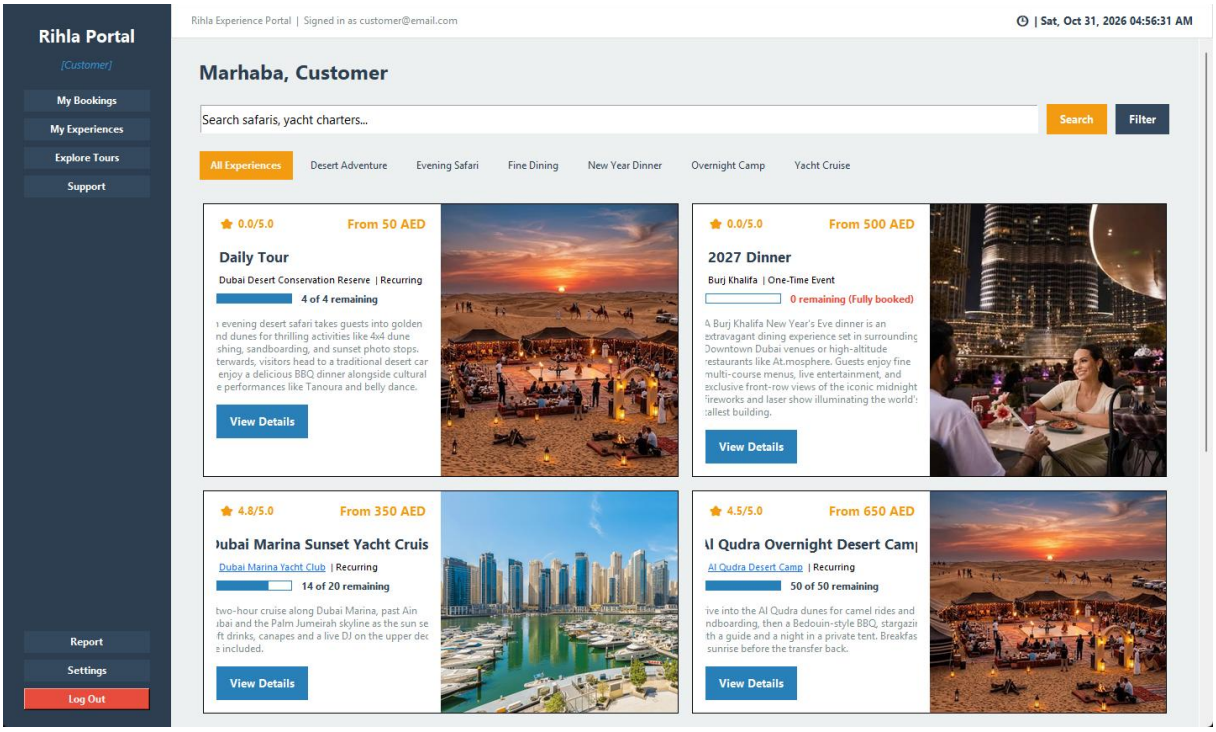


Figure 18: Customer home: Explore Tours with categories, search, ratings and capacity bars

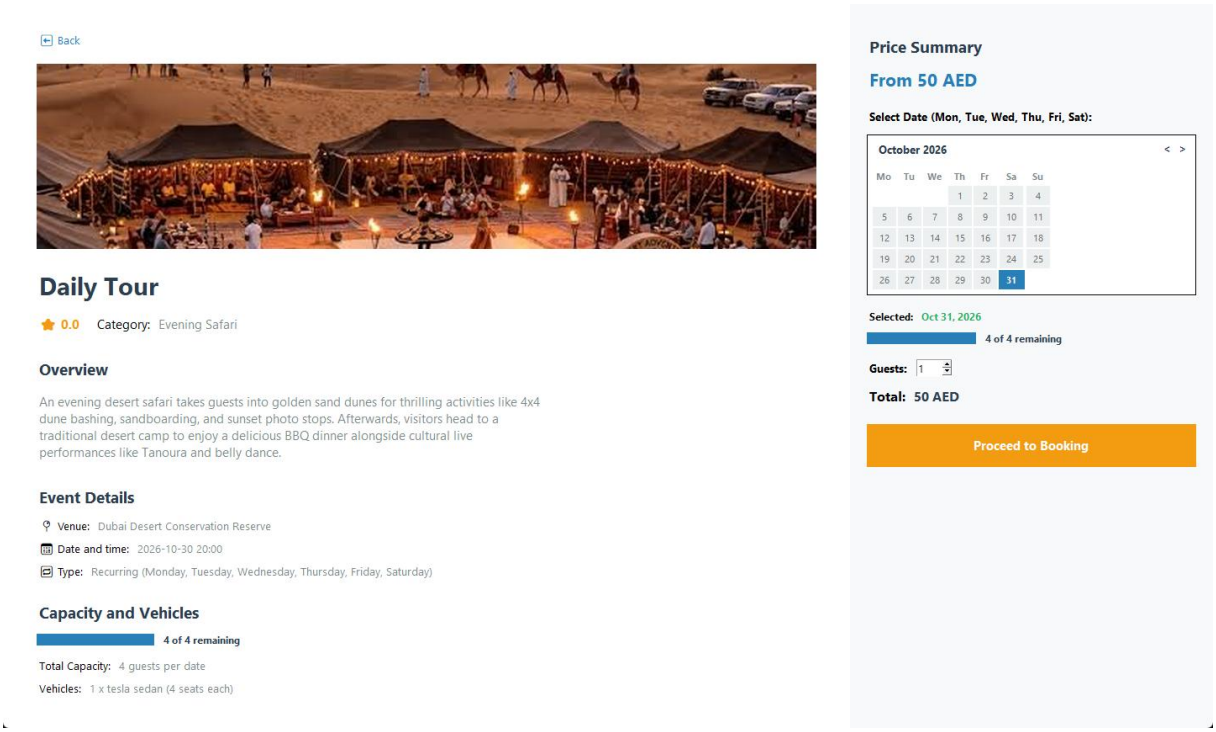


Figure 19: Event page with picture, details, capacity, calendar of allowed dates and price summary

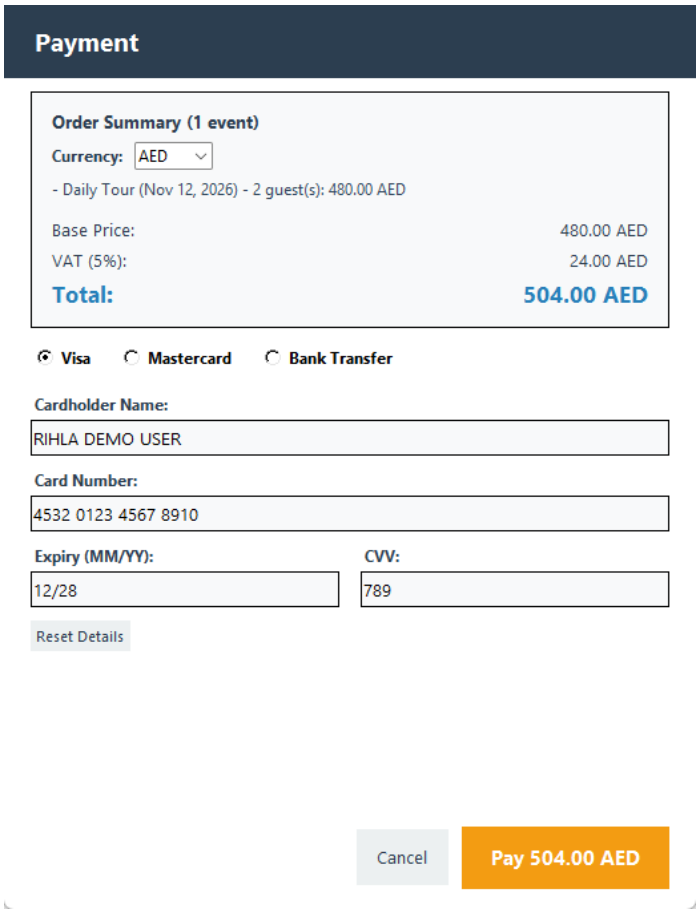


Figure 20: Payment window with currency picker, VAT line and prefilled demo card

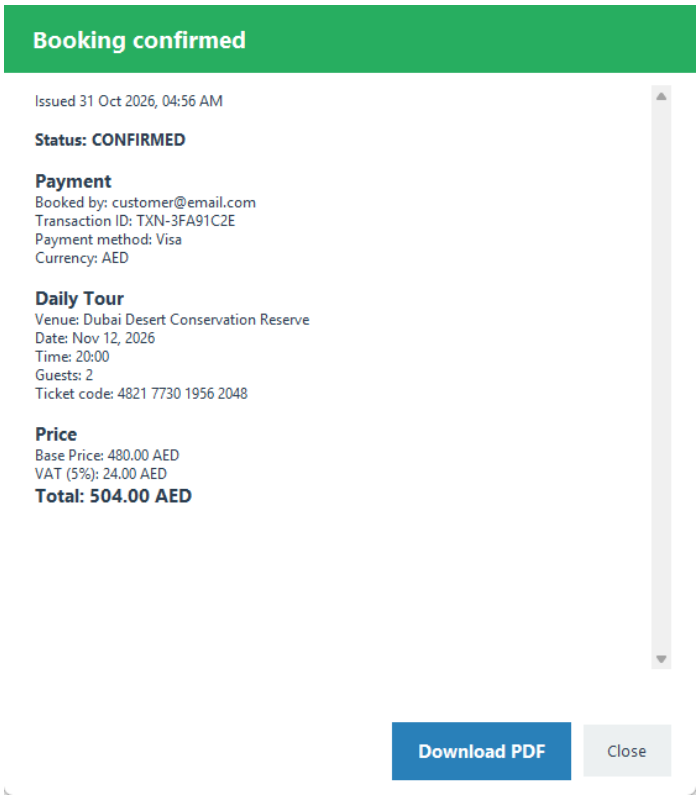


Figure 21: Booking Details window after payment with Download PDF

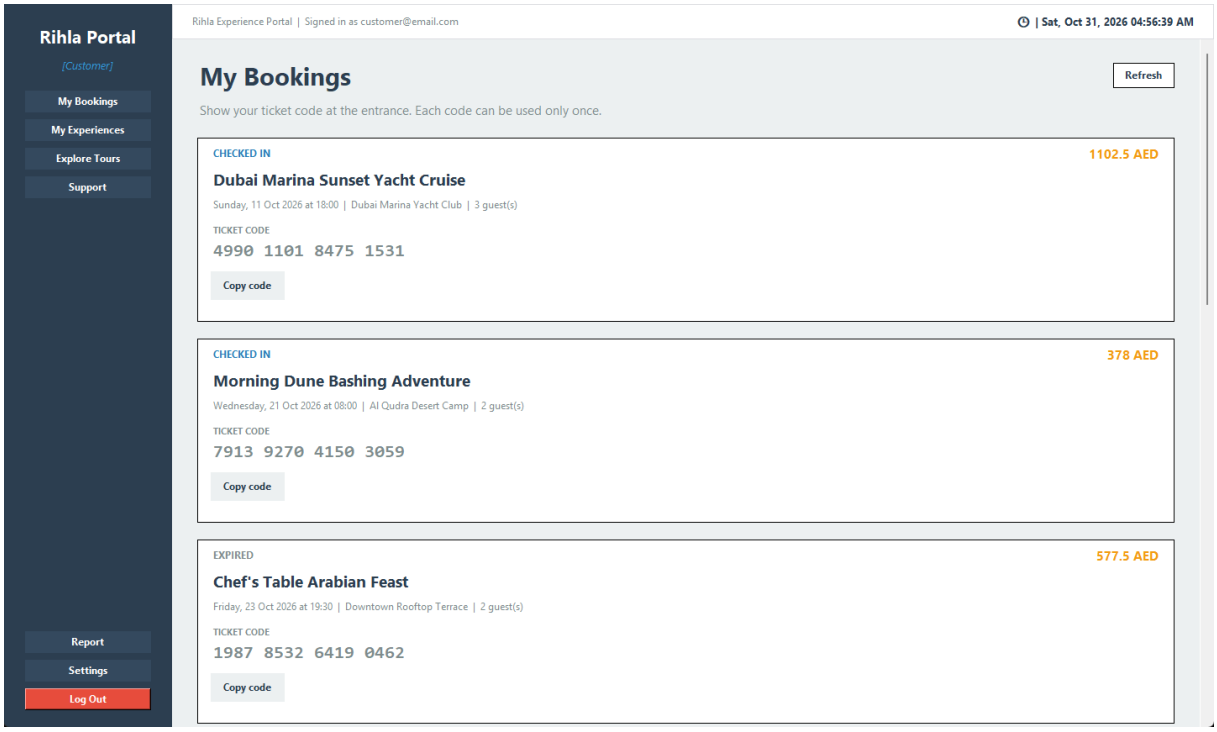


Figure 22: My Bookings with ticket codes, statuses and Copy code buttons

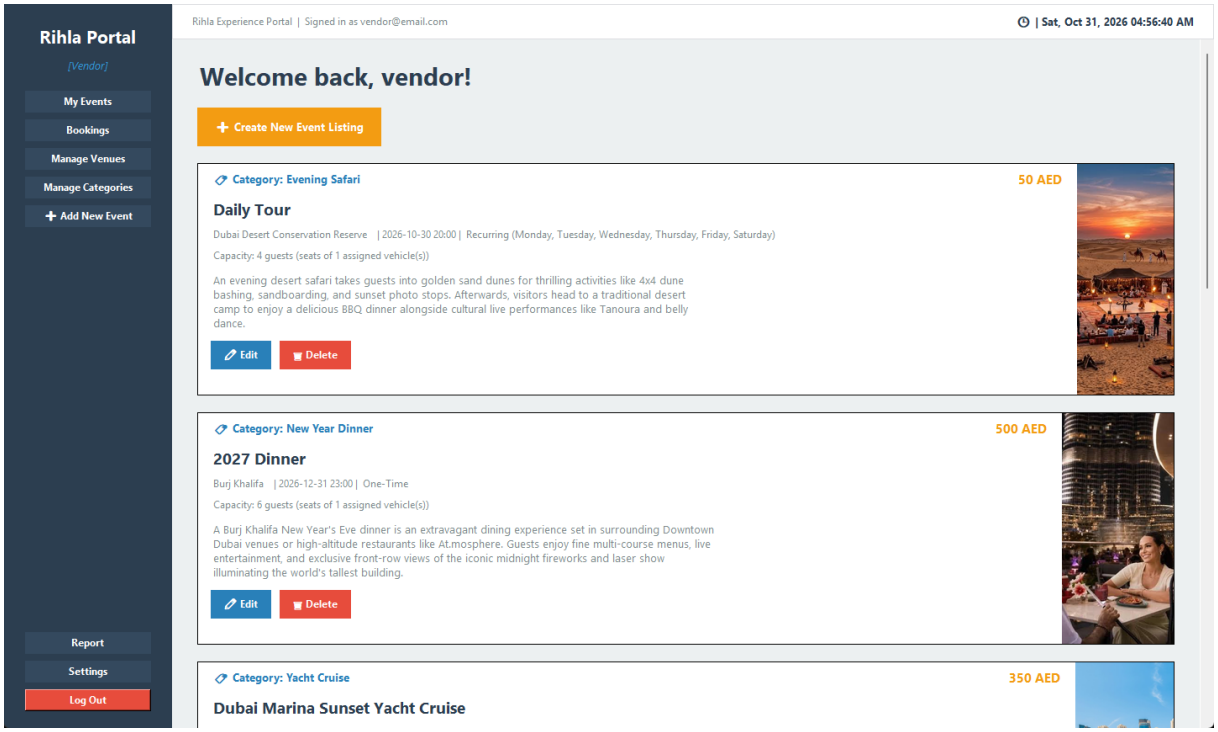


Figure 23: Vendor home: My Events with pictures, Edit and Delete

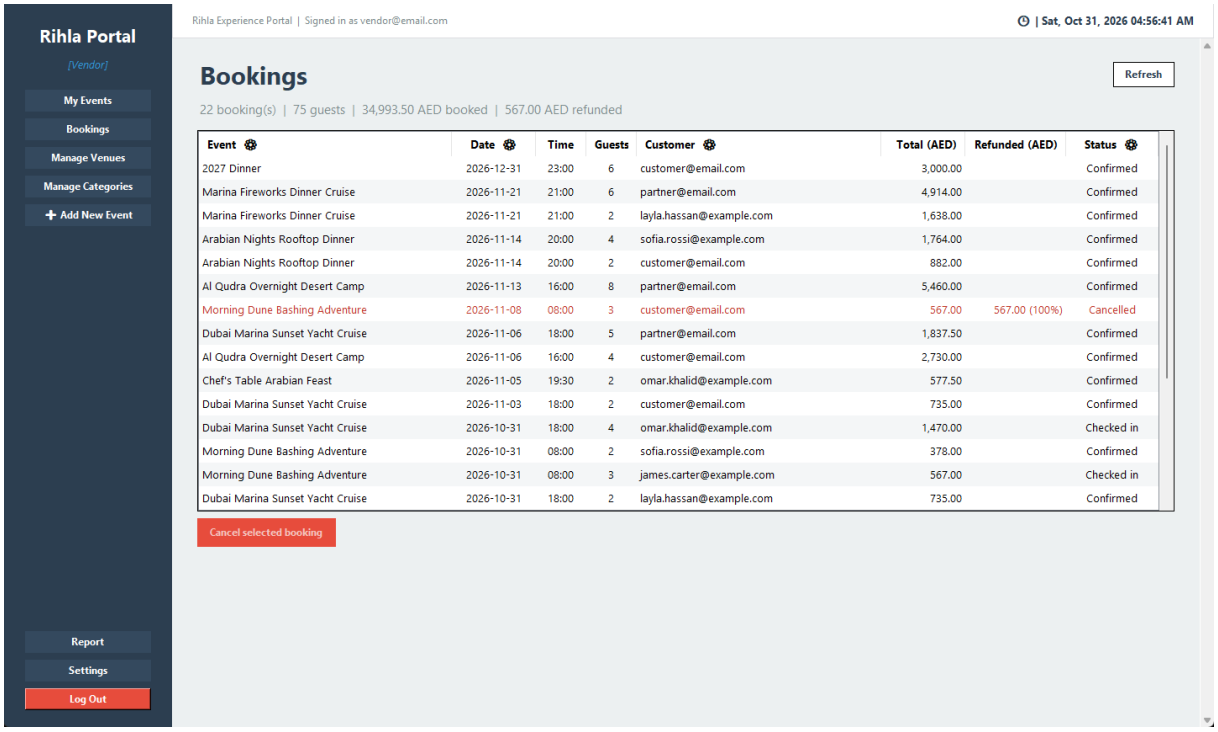


Figure 24: Vendor Bookings table with filterable columns and cancelled rows

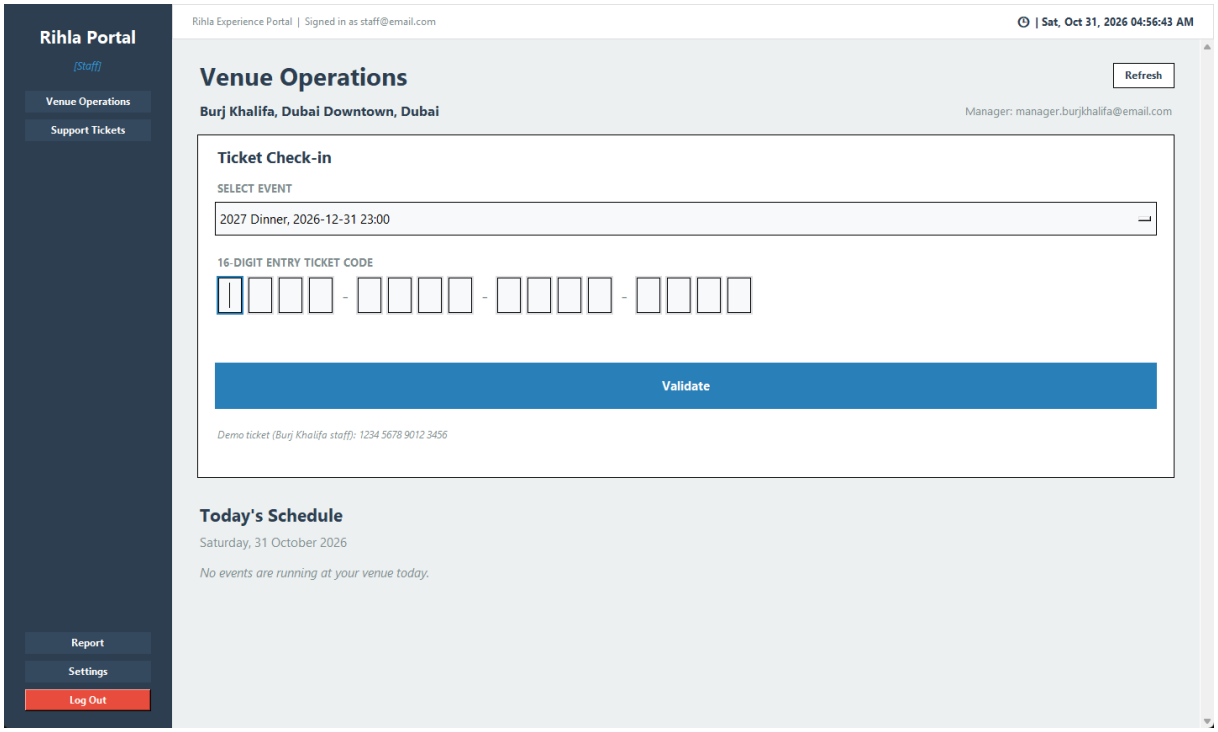


Figure 25: Staff Venue Operations: venue, ticket check-in and the schedule of the day

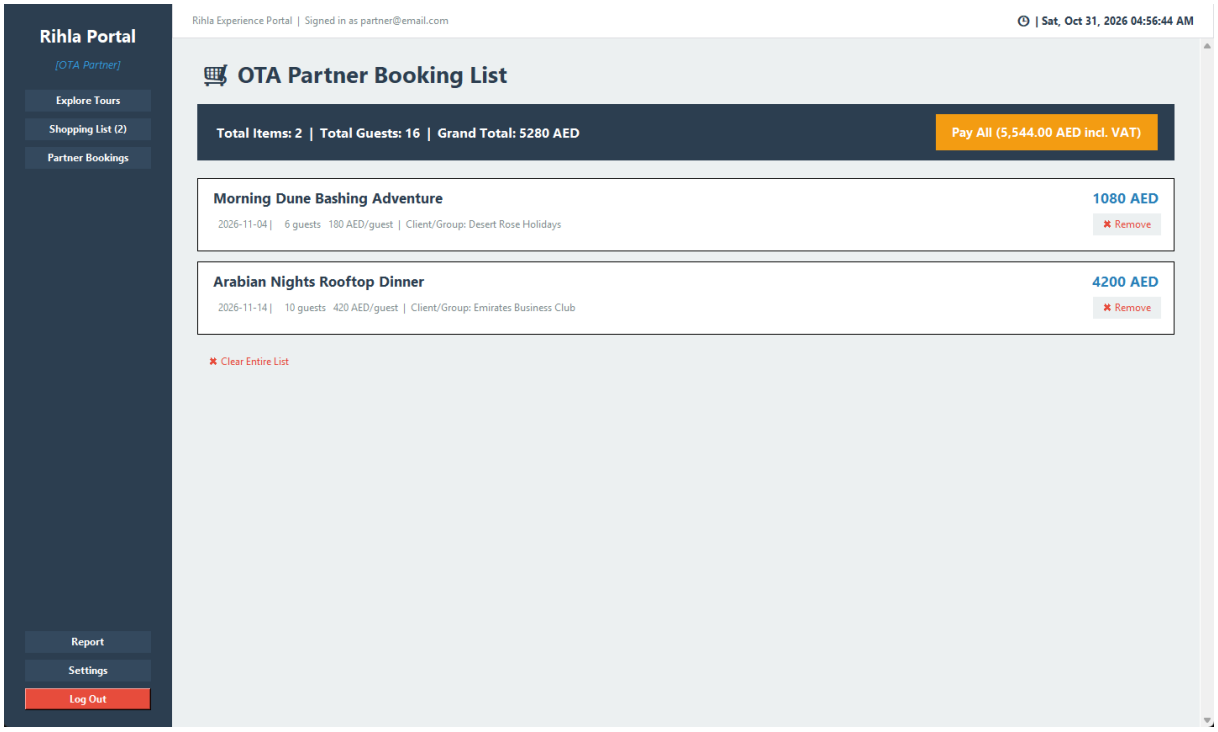


Figure 26: Partner shopping list with one payment for all events

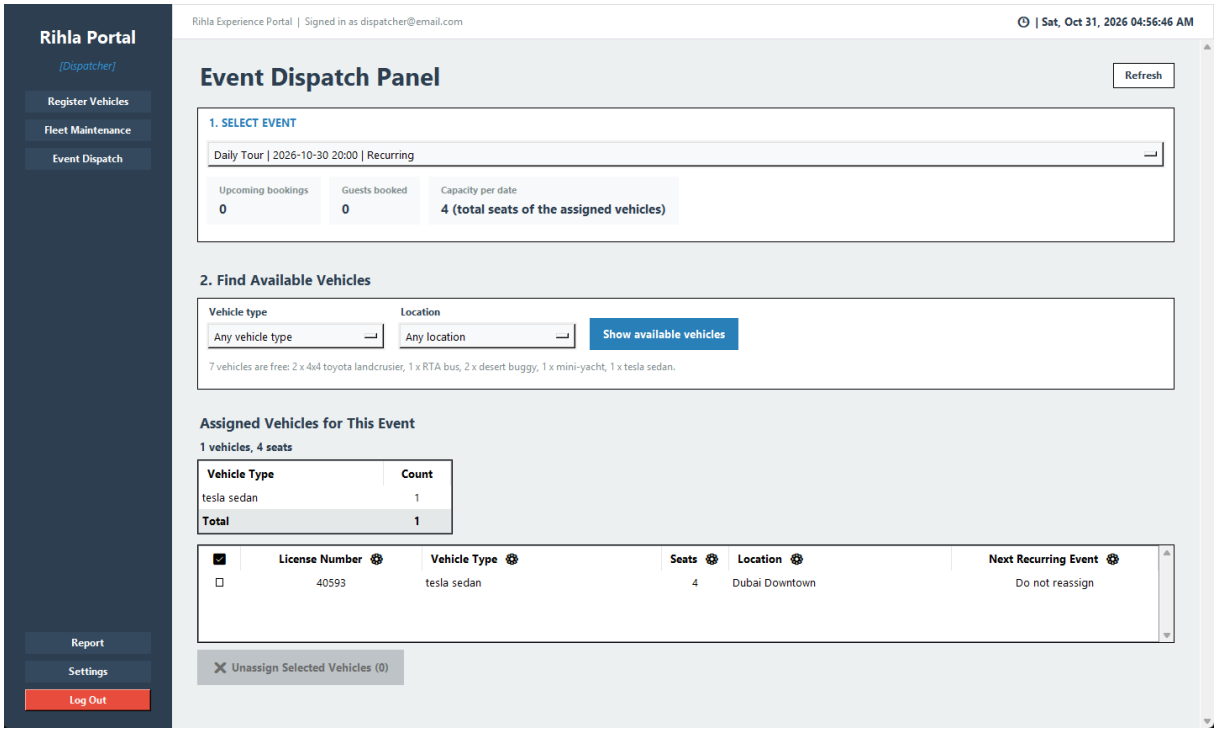


Figure 27: Dispatcher Event Dispatch panel with available and assigned vehicles

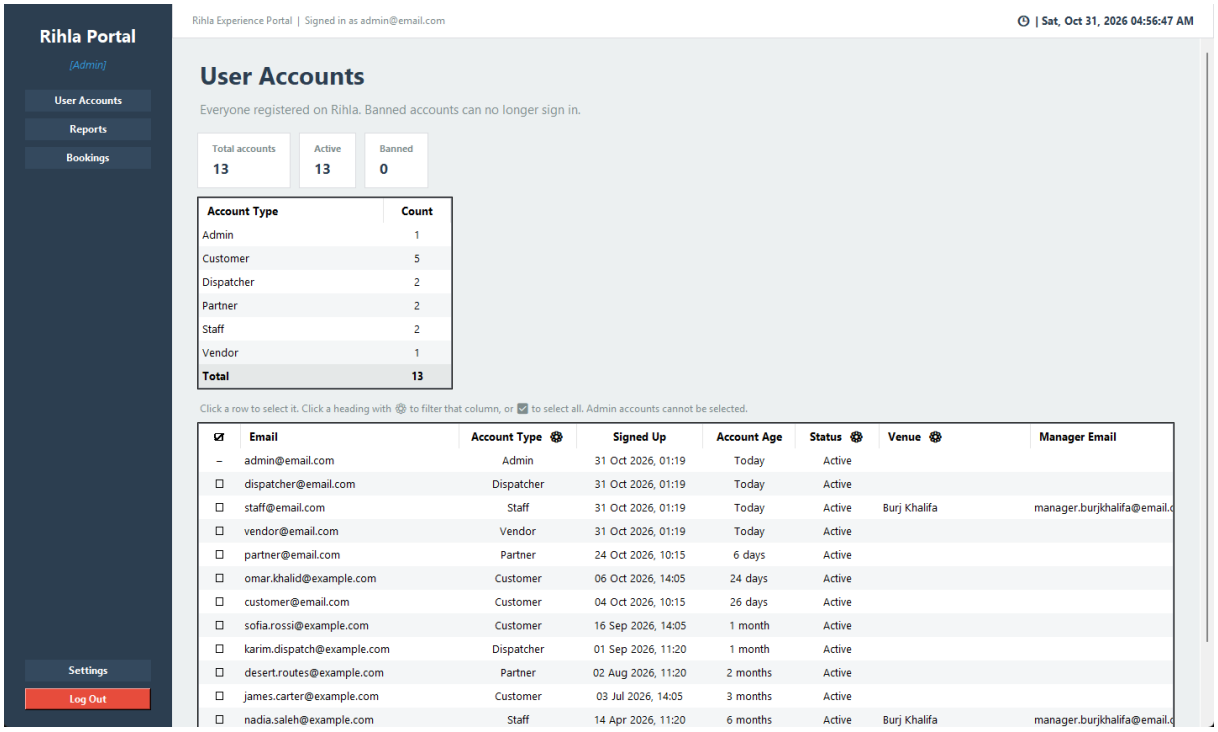


Figure 28: Admin User Accounts with account type counts, sign-up date, age and status

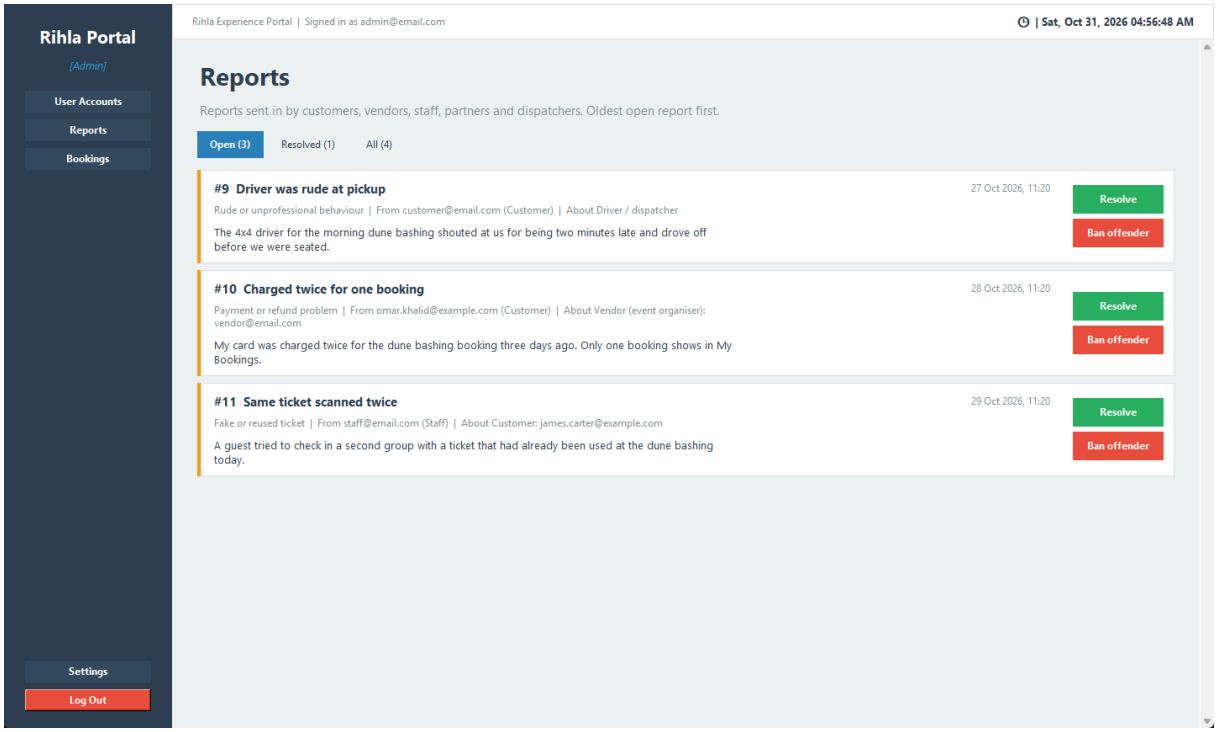


Figure 29: Admin Reports inbox with Resolve and Ban offender buttons

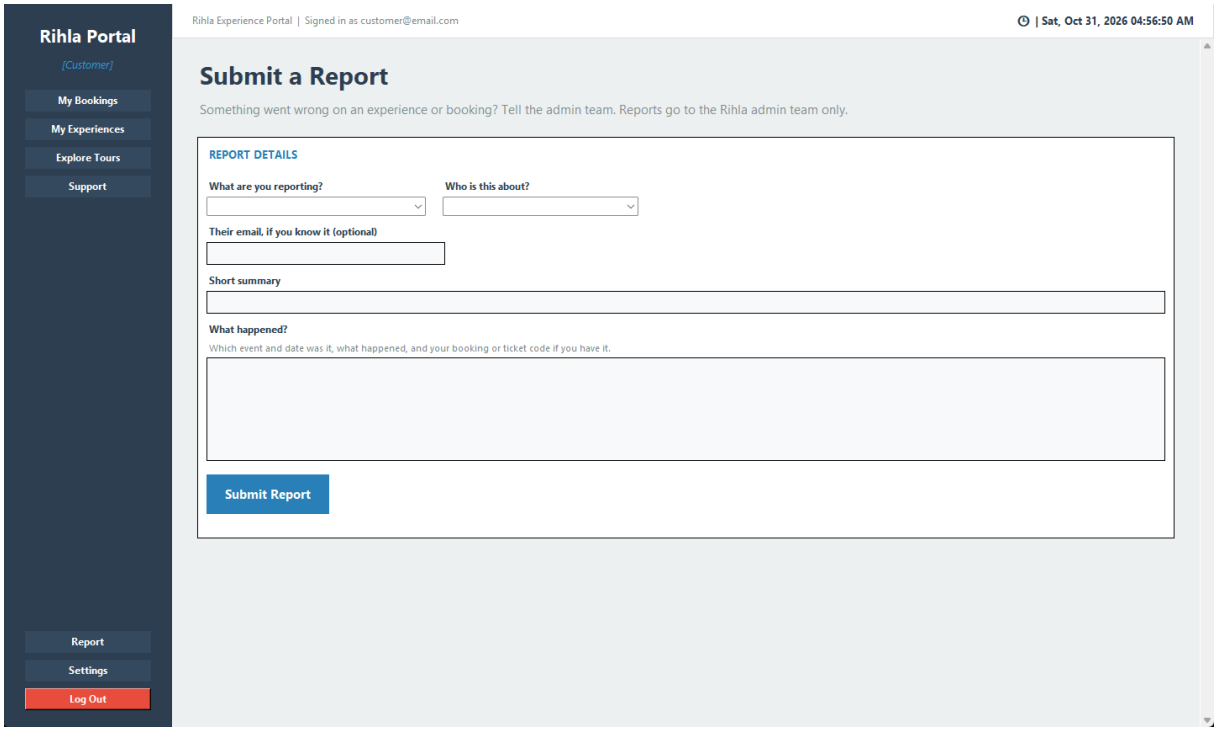


Figure 30: Report form used by every non-admin role

4.1.2 Objects and actions

Screen	Objects	Actions
Login	Email and password fields, eye button, Sign In, Google and Apple buttons, demo chips, links	Sign in, show password, fill a demo account, open sign-up or Forgot password
Create Account	Account type menu, SSN digit boxes, venue menu and manager email (staff), email and phone with Verify buttons, password strength label, Terms check box	Pick role, verify email and phone with code 000000, register
Forgot / Reset password	Email field, Send Code, 6-digit code, new password fields	Send code, verify, save new password
Explore Tours (Customer, Partner)	Search box, category chips, Filter window with price slider, event cards with picture, rating, capacity bar and View Details	Search, filter, open event
Event page	Cover picture, venue link with Open in Google Maps, capacity bar, calendar, guests spinner, client name (partner), price summary	Choose date and guests, Proceed to Booking, Add to List, Pay Now
Payment window	Order summary, currency box, Base Price, VAT and Total, Visa, Mastercard and Bank Transfer options, prefilled fields, Reset Details	Change currency, change method, Pay, Cancel
Booking Details	Read-only receipt text	Download PDF, Close
My Bookings / Partner Bookings	Booking cards with status, ticket code, refund policy line, Refresh	Copy code, Cancel booking
My Experiences	Past bookings with stars	Rate and comment
Support (Customer)	Event menu, subject, message, list of own tickets	Send ticket
Vendor pages	Event cards, event form (picture, recurrence days, capacity), bookings table with gear filters, venues list, categories list	Add, edit, delete events, venues and categories, cancel a booking with a reason
Venue Operations (Staff)	Venue and manager line, event menu, 16 digit boxes, Validate, schedule table	Check a ticket in, refresh
Support Tickets (Staff)	List of open tickets of the venue	Resolve
Dispatcher pages	Vehicle form, fleet table with filters and check boxes, event menu, type and location filters, assigned list	Register, set Maintenance or Available, delete, assign, free
Admin pages	Account type counts, users table, reports inbox, bookings table	Ban, restore, resolve, ban offender, cancel any booking
Settings (all roles)	Preferred currency, staff venue and manager email, change password	Save settings
Report (non-admin)	Category, about whom, offender email, summary, details	Submit report

Table 24: Screen objects and actions

4.2 Interface design rules

We followed Shneiderman's eight golden rules. The table shows where each rule appears in Rihla.

Golden rule	How Rihla follows it
1. Strive for consistency	One frame (BaseWelcomeScreen) for all roles, one colour and font file (theme.py), one table style (table_style.py) with bold headings and a gear on filterable columns.
2. Seek universal usability	Quick demo account chips for new users, prefilled demo payment details for experts and testers, prices in four currencies.

Golden rule	How Rihla follows it
3. Offer informative feedback	Capacity bar with 'N of TOTAL remaining' and 'Hurry up!', strength label for passwords, payment declines shown inside the payment window, clear server messages.
4. Design dialogs to yield closure	After payment the Booking Details window sums up the whole booking with the ticket code and a PDF button.
5. Prevent errors	The calendar only allows valid dates, DigitBoxes only accept digits, the role picker hides fields that do not apply, the server checks everything again.
6. Permit easy reversal of actions	Bookings can be cancelled under a clear policy, cart items can be removed, banned accounts can be restored, Reset Details restores the payment form.
7. Keep users in control	Cancellation shows the refund before the user confirms. Pages that show other people's changes have a Refresh button.
8. Reduce short-term memory load	Ticket codes are grouped in four blocks of four digits with a Copy code button. The top bar always shows the signed-in email.

Table 25: Eight golden rules in Rihla

4.3 Components available

The interface is built only with the standard Tkinter toolkit and two small libraries:

- **tkinter:** Tk, Toplevel, Frame, Label, Button, Entry, Text, Canvas, Scrollbar, Scale, Radiobutton, Checkbutton, OptionMenu and messagebox, filedialog.
- **tkinter.ttk:** Combobox (currency and other choices) and Treeview (all tables).
- **Custom widgets:** DigitBoxes (digit-per-box input for SSN and ticket codes), CapacityBar (seat bar), CoverImage and build_card_shell (event cards with pictures), venue_label (map link).
- **Pillow:** loading, resizing and showing event pictures.
- **tksvg (optional):** draws the Lucide SVG icons. Without it the app still runs with fewer icons.

4.4 UIDS description

We did not use a user interface development system or a GUI builder. All screens are written by hand in Python with Tkinter. To keep the code tidy we use shared modules instead: theme.py for colours and fonts, table_style.py for tables, ui.clear_window to swap screens, navigate.py to move between screens and routing.py to pick the home screen of a role. Adding a role needs one new welcome module and one line in routing.py.

5.0 Restrictions, limitations, and constraints

The following design issues limit the current version. We list them openly because they matter for anyone who wants to run Rihla outside a classroom demo.

- **No login tokens:** the server does not give the client a session token. Requests carry an email, and the server trusts it. The server does check that the email belongs to an active account with the right role (_blocked, is_admin), but a person who sends hand-made HTTP requests could act as another user.
- **Missing ownership checks:** PUT /api/users/password has no check, and event update and delete, venue delete, and the vehicle and dispatch endpoints do not check that the caller owns the record.
- **Local only:** the server listens on 127.0.0.1 port 5000 with plain HTTP and the Flask development server. It is meant for one machine. It is not ready for the internet (no HTTPS, no production server).
- **Demo payments and refunds:** payments only compare the form with fixed demo details. Refunds are stored in the database but no money moves.
- **Fixed currency rates:** USD, EUR and GBP use fixed rates in pricing.py. Everything is charged and stored in AED.
- **Fake verification:** email, phone and password reset codes are always 000000. Google and Apple login is a normal email login.
- **SQLite:** one database file is simple and needs no setup, but it allows only one writer at a time. BEGIN IMMEDIATE makes bookings safe, but many parallel users would wait for each other.
- **Prices as text:** prices are saved as text such as '240 AED' and only the digits are used, so prices with decimals are not supported.
- **Email as link:** many tables point to users by email instead of a numeric key, so an email cannot be changed without updating other tables.
- **Server time:** all times use the local time of the server machine, not a fixed time zone.
- **Features from Phase 1 that were not built:** QR codes, an offline driver app, live vehicle tracking, email and SMS messages, the 15-minute seat lock (replaced by the transaction check) and Arabic language support.
- **Platform:** the client is tested on Windows. Some parts (Downloads folder lookup, zoomed window) have fallbacks for other systems but were not tested there.

6.0 Testing Issues

Rihla has an automated test suite in the tests/ folder that uses Python's unittest. The tests point the database to a temporary file, so they never change server/users.db. They import the server modules directly (tests/db_test_helpers.py) and some tests call the Flask API through app.test_client(). A few tests open Tkinter windows for a short time. There are 70 tests. On 31 October 2026 all 70 passed in about five minutes on a team laptop. The whole suite runs with:

```
python -m unittest discover -s tests
```

6.1 Classes of tests

Test class (file)	Type	What it covers
BookingTests (test_booking.py)	Black-box unit	Price with VAT and ticket code, refusals, booking the same event again, single check-in, cancel stops a ticket, rename and delete event, vendor view, schedule, support tickets
CancellationTests (test_booking.py)	White-box unit	Refund steps at 48 h and 24 h, VAT in proportion, audit trail, vendor and admin cancellation, no cancel after start, event delete refunds, cart rollback
CapacityTests (test_booking.py)	White-box unit	Event capacity, vehicle seats, same vehicle type adds up, freeing a vehicle, capacity per date, bad capacity values, old database upgrade
PartnerAndPaymentTests (test_partner_and_payment.py)	Unit and GUI	CapacityBar text, partner bookings above 10 guests, shopping list and checkout, remove from cart, demo payment success and failure
AdminTests and ReportTests (test_admin.py)	Black-box unit	Admin demo account, no admin sign-up, sign-up time, ban and restore, admin limits, reports flow and rules, old database upgrade
ApiAccountGuardTests (test_admin.py)	Integration (Flask test client)	Banned and unknown accounts cannot book or open tickets, cart and vendor actions need the right role
StaffVenueTests (test_staff_venue.py)	Integration	Staff sign-up rules, venue-scoped check-in, schedule and support tickets, changing venue, banned staff, venue delete protection, admin columns

Table 26: Classes of automated tests

Black-box tests call a public function or endpoint with inputs and check only the result, for example that a booking of 3 guests at 240 AED stores a total of 756 AED. White-box tests are written with the code paths in mind, for example the boundaries of refund_percent_for at exactly 48 and 24 hours and the rollback branch of checkout_cart. Integration tests go through server.py with the Flask test client to check the role guards. Next to the automated tests we ran manual system tests with the per-role demo script in README.md, after seeding with dummy_data.ps1.

6.2 Expected software response

Test case	Input	Expected response
Booking price	3 guests, event price 240 AED	Base 720, VAT 36, total 756 AED stored, 16-digit code
Too many guests (customer)	11 guests	Refused: 'You can book 1 to 10 guests at a time.'
Partner group	Partner books more than 10 guests	Accepted while seats remain
Past date	booking_date yesterday	Refused: date is in the past
Wrong weekday	Recurring Tue and Thu event on a Monday	Refused: not available on that date
Full event	guests more than remaining seats	Refused with remaining and total seats
Vehicle capacity	Two 4x4 Land Cruisers assigned	Total capacity 12, source 'vehicles'
Check-in twice	Same valid code two times	First: access granted, admit N guests. Second: already used
Other venue	Staff of Al Qudra checks the Burj Khalifa demo ticket	Refused: not for an event at your venue
Cancel 3 days before	Customer cancels	100% refund, ticket Cancelled, seats free
Cancel 30 hours before	Customer cancels	50% refund, VAT refunded in proportion
Cancel 10 hours before	Customer cancels	Refused: cancellations close 24 hours before
Vendor cancel without reason	Empty reason	Refused: please give the customer a reason
Cart with one bad item	Checkout where one item has no seats	No booking kept, cart unchanged
Changed card number	Payment form edited	Payment declined, shown in the payment window
Sign-up as Admin	role = Admin	Saved as Customer at most, never as Admin
SSN rules	Vendor with 15 digits or 16 equal digits	Refused with a clear message
Banned user	Login and booking after ban	Login 403, booking request 403
Admin ban self	Admin bans own email	Refused: you cannot ban your own account
Review rating	rating 4.3	Refused: steps of 0.5

Table 27: Expected responses for the main test cases

6.3 Performance bounds

- One booking, cancellation or check-in is a single short SQLite transaction. On the team's laptops the answer appears without a visible delay.
- The client waits at most 10 seconds for any HTTP request (timeout in api_client.py) and then tells the user that the server cannot be reached.
- Event pictures are at most 1400 pixels on the long side after upload and at most 4 MB on the server.
- Limits that protect the system: 10 guests per customer booking, 5000 seats per event, 3 open support tickets per customer, 5 open reports per user, 300 characters per cancel reason, 500 per review comment.
- The full test suite takes about five minutes because some tests open Tkinter windows and PBKDF2 hashing is slow on purpose.

6.4 Identification of critical components

These parts need the most attention in testing because a mistake there costs money, seats or trust:

- **_book and _capacity_info:** they decide if a seat can be sold. A bug here means double booking or empty seats.
- **_cancel, refund_percent_for and split_refund:** they decide how much money a customer gets back. The hour boundaries (48 and 24) and the VAT split must be exact.
- **checkout_cart:** it must keep all bookings of a cart or none of them.
- **staff_verify_ticket:** it must admit a ticket once and only at the right venue.
- **login_status, _blocked and set_user_banned:** they keep banned and unknown accounts out.
- **ssn_error and register_user:** they enforce the SSN rule and keep the raw SSN out of the database.
- **init_db:** it upgrades older databases. The tests create an old database on purpose and check the upgrade.